

ISPGAYA

instituto superior politécnico

Licenciatura em Engenharia Informática
Projeto de Engenharia Informática em Contexto Empresarial
2025/2026

Gabriel Jesus da Rocha

PLATAFORMA UNIFICADA METEOROLÓGICA INTEGRAÇÃO DE MÚLTIPLAS APIs LOCAIS NUMA APLICAÇÃO ESCALÁVEL

Relatório de estágio/projeto orientado pelo Professor Doutor José Carlos Arnaud
apresentada à Escola Superior de Ciência e Tecnologia

Julho de 2026

Dedicatória

Dedico este trabalho aos meus pais e familiares, pelo amor incondicional, pela paciência infinita e por serem o meu porto seguro em todas as etapas desta caminhada. O vosso apoio e a vossa crença inabalável no meu potencial foram o combustível que me permitiu superar os momentos mais difíceis e alcançar esta meta. Esta conquista também é vossa.

Agradecimentos

Gostaria de expressar o meu profundo agradecimento ao meu orientador, o Professor José Arnaud, pela paciência, partilha de conhecimento e rigor científico que demonstrou ao longo de todo o desenvolvimento deste projeto. A sua orientação foi fundamental para a concretização deste trabalho. Agradeço também ao ISPGAYA e ao corpo docente do curso de Engenharia Informática por todas as ferramentas e oportunidades de aprendizagem partilhadas. Aos meus colegas e amigos, agradeço o companheirismo, as discussões de ideias e o apoio mútuo nas longas jornadas de estudo. Por fim, um agradecimento especial à minha família e àqueles que me são mais próximos, pelo suporte emocional, compreensão e incentivo constante. Sem o vosso amor e estabilidade, esta caminhada teria sido muito mais difícil.

Resumo

O presente relatório descreve o desenvolvimento do sistema **Local Weather**, uma plataforma de agregação de APIs meteorológicas nacionais construída como projeto de final de curso de Engenharia Informática. A motivação para o projeto surgiu da constatação de que o utilizador que consulta habitualmente a API do seu país de residência se confronta, ao precisar de previsões para outros países, com a necessidade de recorrer a serviços globais genéricos de menor precisão, por desconhecer a existência das agências meteorológicas nacionais de cada região ou por as encontrar redigidas na língua local.

O sistema integra quatro fornecedores de dados meteorológicos com cobertura geográfica distinta: o IPMA (Portugal), a AEMET (Espanha), a CWA (Taiwan) e o Open-Meteo (cobertura global de quinze países). O *backend* foi desenvolvido em Python com o framework FastAPI, seguindo o padrão arquitetural *Provider Adapter*: cada fornecedor está encapsulado num módulo independente que implementa a lógica de comunicação, autenticação e transformação de dados específica dessa fonte, sem dependências cruzadas entre fornecedores. O sistema expõe treze *endpoints* REST documentados automaticamente via OpenAPI/Swagger UI. O *frontend* foi desenvolvido em React, com uma interface minimalista que inclui pesquisa unificada por cidade com resultados em tempo real de todos os fornecedores, seleção automática do fornecedor com base no país da cidade pesquisada, geolocalização automática, histórico de pesquisas persistido em *localStorage*, suporte a dois idiomas (inglês e português), unidades de medida configuráveis (°C/°F e km/h ou mph), e visualização gráfica da previsão de temperatura.

A qualidade do código foi validada por uma suite de 196 testes automatizados unitários e de integração com 89% de cobertura de código, sem qualquer chamada HTTP real às APIs dos fornecedores durante a execução dos testes.

Os principais desafios técnicos encontrados foram a codificação ISO-8859-1 das respostas AEMET, o protocolo de dois passos desta mesma API, a ausência de um identificador de *dataset* global funcional na plataforma CWA, e as restrições CORS na geocodificação inversa por GPS, todos resolvidos e documentados no presente relatório.

Palavras-chave: agregação de APIs meteorológicas, provider adapter, FastAPI, React, dados abertos, IPMA, AEMET, CWA, Open-Meteo, previsão meteorológica.

Abstract

This report describes the development of ***Local Weather***, a national meteorological API aggregation platform built as a final-year Software Engineering project. The motivation for the project arose from the observation that users who routinely consult their own country's meteorological service face, when seeking forecasts for other countries, the need to resort to less accurate global services either because they are unaware of each region's national meteorological agency, or because those agencies' portals are written in the local language.

The system integrates four weather data providers with distinct geographic coverage: IPMA (Portugal), AEMET (Spain), CWA (Taiwan), and Open-Meteo (global coverage for fifteen countries). The backend was built in Python using the FastAPI framework, following the Provider Adapter architectural pattern: each provider is encapsulated in an independent module that implements the communication, authentication, and data transformation logic specific to that source, with no cross-provider dependencies. The system exposes thirteen REST endpoints with automatic documentation via OpenAPI/Swagger UI. The frontend was developed in React, with a minimalist interface featuring unified city search with real-time results from all providers, automatic provider selection based on the searched city's country, GPS-based geolocation, a search history persisted in localStorage, support for two languages (English and Portuguese), configurable units of measurement (°C/°F and km/h ou mph), and graphical temperature forecast visualisation.

Code quality was validated by a suite of 196 automated tests unit and integration achieving 89% code coverage, with no real HTTP calls to provider APIs during test execution.

The main technical challenges encountered were the ISO-8859-1 encoding of AEMET responses, the two-phase request protocol of the same API, the absence of a functional global dataset identifier in the CWA platform, and CORS restrictions in GPS-based reverse geocoding all resolved and documented in this report.

Keywords: meteorological API aggregation, provider adapter, FastAPI, React, open data, IPMA, AEMET, CWA, Open-Meteo, weather forecasting.

Lista de abreviaturas e siglas

4D-Var	Assimilação variacional 4-dimensional
AEMET	<i>Agencia Estatal de Meteorología</i> (Espanha)
AIFS	<i>Artificial Intelligence Forecasting System</i>
ALADIN	Modelo regional de previsão europeu
API	<i>Application Programming Interface</i>
AROME	<i>Application of Research to Operations at Mesoscale</i>
ARW-WRF	<i>Advanced Research WRF</i>
ASGI	<i>Asynchronous Server Gateway Interface</i>
BTABoK	<i>Business Technology Architecture Body of Knowledge</i>
CORS	<i>Cross-Origin Resource Sharing</i>
CU	Caso de Uso
CWA	<i>Central Weather Administration</i> (Taiwan)
DOM	<i>Document Object Model</i>
DWD	<i>Deutscher Wetterdienst</i> (Alemanha)
ECMWF	<i>European Centre for Medium-Range Weather Forecasts</i>
ENS	<i>Ensemble</i> (previsão probabilística ECMWF)
ERA5	<i>ECMWF Reanalysis v5</i>
FMI	<i>Finnish Meteorological Institute</i>
GFS	<i>Global Forecast System</i> (EUA)
GoF	<i>Gang of Four</i>
GPS	<i>Global Positioning System</i>
HATEOAS	<i>Hypermedia as the Engine of Application State</i>
HIRLAM	<i>High Resolution Limited Area Model</i>
HRES	<i>High Resolution</i> (componente determinístico do IFS)
HTTP	<i>Hypertext Transfer Protocol</i>
ICON	<i>Icosahedral Nonhydrostatic</i> (modelo do DWD)
IFS	<i>Integrated Forecasting System</i> (ECMWF)

INE	<i>Instituto Nacional de Estadística (Espanha)</i>
INMET	Instituto Nacional de Meteorologia (Brasil)
IPMA	Instituto Português do Mar e da Atmosfera
JSON	<i>JavaScript Object Notation</i>
NCEP	<i>National Centers for Environmental Prediction</i>
NWP	<i>Numerical Weather Prediction</i>
OGC	<i>Open Geospatial Consortium</i>
PWA	<i>Progressive Web App</i>
REST	<i>Representational State Transfer</i>
RF	Requisito Funcional
RNF	Requisito Não-Funcional
SOA	<i>Service-Oriented Architecture</i>
TTL	<i>Time To Live</i>
TWRF	<i>Typhoon WRF</i>
URI	<i>Uniform Resource Identifier</i>
URL	<i>Uniform Resource Locator</i>
WFS	<i>Web Feature Service</i>
WIS	<i>WMO Information System</i>
WMO	<i>World Meteorological Organization</i>
WSGI	<i>Web Server Gateway Interface</i>
XML	<i>eXtensible Markup Language</i>

Índice

Dedicatória.....	iii
Agradecimentos.....	iv
Resumo	v
Abstract.....	vi
Lista de abreviaturas e siglas.....	vii
1. Introdução.....	13
1.1. Enquadramento.....	13
1.2. Objetivos.....	14
1.3. Apresentação do Projeto	15
1.4. Estrutura do Relatório	16
2. Fundamentação Teórica.....	17
2.1. Previsão Meteorológica Numérica	17
2.1.1. Modelos Globais.....	18
2.1.2. Modelos Regionais e Locais	18
2.2. A Inteligência Artificial em Previsão Meteorológica.....	19
2.3. Dados Abertos Meteorológicos e APIs Nacionais	20
2.3.1. O Movimento de Dados Abertos em Meteorologia.....	20
2.3.2. Os Fornecedores Integrados no Presente Sistema	21
2.4. Padrões de Arquitetura de <i>Software</i>	22
2.4.1. O padrão <i>Aggregator e API Gateway</i>	23
2.4.2. Arquitetura REST	23
2.5. Tecnologias de Implementação	24
2.5.1. FastAPI.....	24
2.5.2. HTTPX e Comunicação Assíncrona	25
2.5.3. React.....	25
2.5.4. Geocodificação e APIs de Geolocalização	26
2.5.5. OpenAPI e Documentação Automática	26
3. Metodologia	27
3.1. Planeamento e Organização	27
3.1.1. Seleção dos Fornecedores de Dados	28
3.1.2. Desenvolvimento e Controlo de Versões.....	28

3.1.3.	Estratégia de Testes	28
4.	Análise e Requisitos.....	29
4.1.	Requisitos Funcionais	29
4.1.1.	Backend	29
4.1.2.	Frontend.....	31
4.2.	Requisitos Não-Funcionais.....	32
4.3.	Casos de Uso	34
4.3.1.	CU01 - Pesquisar Cidade e Consultar Previsão	34
4.3.2.	CU02 - Usar Geolocalização Automática.....	35
4.3.3.	CU03 — Repetir Pesquisa Recente	35
4.3.4.	CU04 — Configurar Unidades e Preferências	36
4.3.5.	Consultar a API via Swagger UI	37
4.4.	Restrições e Limitações Identificadas	37
4.4.1.	Open-Meteo.....	37
4.4.2.	IPMA.....	38
4.4.3.	AEMET	38
4.4.4.	CWA.....	38
5.	Arquitetura e Implementação.....	39
5.1.	Visão e Arquitetura Geral	39
5.2.	<i>Backend</i> – Estrutura dos Módulos.....	40
5.3.	Adaptadores por Fornecedor	41
5.3.1.	Open-Meteo	41
5.3.2.	IPMA.....	41
5.3.3.	AEMET	42
5.3.4.	CWA Taiwan	43
5.3.5.	Geocodificação e Geolocalização.....	44
5.4.	Frontend - Arquitetura React.....	44
5.4.1.	Pesquisa Unificada de Cidades.....	44
5.4.2.	Seleção do Fornecedor e Obtenção de Dados.....	45
5.4.3.	Estado Global - Contextos React	46
5.4.4.	Visão Gráfica	46
5.5.	Comunicação Frontend-Backend.....	46
5.6.	Principais Desafios de Implementação	47
5.6.1.	Codificação ISO-8859-1 das Respostas AEMET	47

5.6.2.	Identificadores de Dataset da CWA	47
5.6.3.	CORS na Geocodificação Inversa.....	48
5.6.4.	<i>Mocking</i> Assíncrono nos Testes	48
6.	Testes e Validação	48
6.1.	Estratégia de Testes	48
6.1.1.	Ausência de Chamadas HTTP Reais.....	49
6.1.2.	Padrão Correto de <i>Mock</i> do httpx	49
6.1.3.	Regra do <i>Patch</i> no Local de Uso.....	50
6.2.	Testes Unitários	50
6.2.1.	Geocodificação (test_geocoding.py).....	50
6.2.2.	Fornecedor AEMET (test_aemet.py).....	51
6.2.3.	Fornecedor IPMA (test_ipma.py).....	52
6.2.4.	Fornecedor CWA (test_cwa.py)	52
6.3.	Testes de Integração.....	54
6.3.1.	Testes de Saúde e Infra-estrutura (<i>TestHealth</i>)	54
6.3.2.	Testes Open-Meteo (TestOpenMeteoEndpoints).....	54
6.3.3.	Testes IPMA (TestIPMAEndpoints)	55
6.3.4.	Testes AEMET (TestAEMETEndpoints).....	55
6.3.5.	Testes CWA (TestCWAEndpoints)	55
6.4.	Resultados e Cobertura	55
6.4.1.	Sumário de Resultados.....	56
6.4.2.	Cobertura por Módulo	56
6.4.3.	Análise da Cobertura.....	57
7.	Conclusões e Trabalho Futuro.....	58
7.1.	Limitações Atuais	58
7.2.	Trabalho Futuro	59
7.2.1.	<i>Endpoint</i> Unificado no <i>Backend</i>	59
7.2.2.	<i>Caching</i> de Respostas com TTL	60
7.2.3.	Expansão de Fornecedores	60
7.2.4.	Observabilidade e <i>Dashboard</i> de Saúde dos Fornecedores	61
7.2.5.	Comparação de Cidades.....	61
7.2.6.	Análise de Precisão Histórica	61
7.2.7.	Aplicação Progressiva (<i>Progressive Web App</i>)	62
8.	Referências Bibliográficas	63

9. Apêndices	68
--------------------	----

Índice de apêndice/anexo

Apêndice A – Website Documentação URL	68
Apêndice B - Estrutura de ficheiros do <i>backend</i>	68
Apêndice C - Estrutura de ficheiros do <i>frontend</i>	69

Índice de figuras

Figura 1 - Fluxo de comunicação entre os componentes do sistema	39
Figura 2 - Fluxo do endpoint "Condições atuais"	41
Figura 3 - IPMA <i>adapter</i>	42
Figura 4 - AEMET fluxo de duas etapas	42
Figura 5 - Descodificação ISO-8859-1	43
Figura 6 - JSON comunicação <i>frontend</i> com <i>backend</i>	47
Figura 7 - Proteção de <i>mock</i> do httpx	49
Figura 8 - Teste da função <i>_decode()</i>	51
Figura 9 - Teste de <i>datasetsIds</i>	53
Figura 10 - Teste de endpoints Open-Meteo	54

Índice de tabelas

Tabela 1 - Requisitos funcionais do <i>backend</i>	29
Tabela 2 - Requisitos Funcionais do <i>Frontend</i>	31
Tabela 3 - Requisitos Não-Funcionais do Sistema	32
Tabela 4 - Caso de uso CU01 - Pesquisar Cidade e	34
Tabela 5 - Caso de uso CU02 - Usar Geolocalização	35
Tabela 6 - Caso de uso CU03 - Repetir Pesquisa Recente	35
Tabela 7 - Caso de uso CU04 — Configurar Unidades e Preferências	36
Tabela 8 - Caso de uso CU05 - Consultar a API via	37
Tabela 9 - Sumário de testes	56
Tabela 10 - Cobertura de código por módulo do <i>backend</i>	56

1. Introdução

1.1. Enquadramento

A previsão meteorológica constitui um dos pilares mais críticos para a tomada de decisão em domínios tão diversos como a aviação, a agricultura, a gestão de energias renováveis, a logística e a segurança pública. Ao longo das últimas décadas, o desenvolvimento de modelos numéricos de previsão do tempo (*Numerical Weather Prediction*, NWP) tem permitido avanços significativos na qualidade das previsões disponíveis ao público em geral (Bi et al., 2023a). Contudo, existe uma tensão fundamental entre dois tipos de abordagens: os modelos globais, que cobrem todo o planeta mas operam a resoluções espaciais relativamente grosseiras (tipicamente 9 a 13 km), e os modelos locais ou regionais, que operam a resoluções inferiores a 5 km e, por isso, produzem previsões mais precisas para a área que cobrem (Frnda et al., 2022).

Neste contexto, diversas agências meteorológicas nacionais desenvolveram e disponibilizaram publicamente as suas próprias *Application Programming Interfaces* (APIs), que expõem os dados dos seus modelos locais em formato acessível por máquina. A Agência Portuguesa do Mar e da Atmosfera (IPMA) (Instituto Português do Mar e da Atmosfera (IPMA), 2024a), a agência espanhola *Agencia Estatal de Meteorología* (AEMET) (Agencia Estatal de Meteorología (AEMET), 2024), a *Central Weather Administration* (CWA) de Taiwan (Central Weather Administration (CWA), 2024), ou o serviço global Open-Meteo (Zippenfenig, 2024) são exemplos concretos desta tendência. No entanto, a ausência de um padrão comum de dados, a diversidade de protocolos de autenticação, as barreiras linguísticas dos portais de documentação e a heterogeneidade dos modelos de resposta JSON (*Javascript Object Notation*), constituem obstáculos significativos para qualquer programador que pretenda integrar múltiplas fontes num único sistema coerente. (Melgarejo-Hernández et al., 2026)

A motivação para o desenvolvimento deste projeto tem origem numa experiência quotidiana concreta: o utilizador final que consulta habitualmente o IPMA para acompanhar as condições meteorológicas em Portugal confronta-se, ao querer obter previsões para outros países, seja para acompanhar familiares dispersos pelo mundo, seja no contexto de viagens internacionais com a necessidade de recorrer a serviços globais genéricos, por desconhecer a existência ou a localização das agências meteorológicas nacionais de cada país. Esta dificuldade é agravada pelo facto de os portais dessas agências se encontrarem frequentemente redigidos na

língua local, tornando a sua utilização inacessível para um utilizador sem conhecimento do idioma. O presente projeto parte precisamente desta lacuna para propor uma solução que centraliza o acesso aos principais serviços meteorológicos nacionais numa única plataforma, em língua acessível e com uma interface uniforme.

A escolha do tema assenta ainda na convicção de que, tal como acontece em outros domínios de agregação de dados, nomeadamente, os sistemas de *API Gateway* e o padrão *Aggregator* em arquiteturas de micro-serviços (IASA Global, 2023; TatvaSoft, 2023), é possível abstrair a complexidade de múltiplos fornecedores de dados meteorológicos por detrás de uma interface unificada, sem perder a precisão inerente a cada fonte local.

1.2. Objetivos

O presente projeto tem como objetivo principal o desenvolvimento de um sistema full-stack composto por um *backend Representational State Transfer* (REST) e por uma interface *web* que agregue dados meteorológicos provenientes de múltiplos fornecedores nacionais e internacionais, abstraindo as diferenças entre os seus modelos de dados e protocolos de acesso. De forma mais concreta, o projeto visa alcançar os seguintes objetivos específicos:

- **Identificar e integrar APIs meteorológicas nacionais** - selecionar um conjunto representativo de fornecedores com cobertura geográfica distinta (Portugal, Espanha, Taiwan e cobertura global), analisar as respetivas documentações e implementar adaptadores para cada um deles;
- **Conceber uma arquitetura de *provider adapter*** - desenvolver o *backend* segundo o padrão *Adapter* (Escrich, 2023), garantindo o isolamento entre fornecedores e permitindo a adição futura de novas fontes sem alteração das interfaces existentes;
- **Expor uma API REST documentada** - disponibilizar um conjunto de *endpoints* RESTful com documentação automática via OpenAPI/Swagger, que permita a terceiros compreender e utilizar o sistema sem necessidade de conhecer os fornecedores subjacentes;
- **Desenvolver uma interface web orientada ao utilizador** - construir um *frontend* em React que apresente os dados de forma visualmente clara, com suporte a múltiplas unidades de medida, pesquisa unificada por cidade, geolocalização automática e histórico de pesquisas recentes;

- **Garantir a qualidade do código** - implementar um conjunto de testes que cubra as camadas de *parsing*, integração com os fornecedores (via *mocking*) e comportamento dos *endpoints*.

Relativamente ao grau de consecução dos objetivos enunciados, todos os pontos acima foram atingidos. O sistema integra quatro fornecedores (IPMA, AEMET, CWA e Open-Meteo), expõe treze *endpoints* REST documentados, via Swagger UI, e disponibiliza uma interface web com as funcionalidades de pesquisa por cidade, geolocalização automática, tema escuro/claro, unidades configuráveis (°C/°F e km/h ou mph), histórico de pesquisas recentes e visualização gráfica da previsão de temperatura.

1.3. Apresentação do Projeto

O **Local Weather** é um sistema de agregação de APIs meteorológicas desenvolvido na íntegra no âmbito deste projeto. Não se trata de uma aplicação comercial existente, mas antes de um artefacto de engenharia criado com o propósito explícito de demonstrar as competências adquiridas ao longo do curso.

O sistema é composto por dois subsistemas principais:

- **Backend** - desenvolvido em Python com o *framework* FastAPI (Ramírez, 2024), expondo uma API REST com documentação automática OpenAPI. O *backend* é responsável pela comunicação com os fornecedores externos, pela transformação dos dados para modelos internos tipificados (via **Pydantic**) e pela geocodificação de nomes de cidades em coordenadas geográficas;
- **Frontend** - desenvolvido em React, com uma interface minimalista orientada à usabilidade. O frontend inclui lógica de seleção do fornecedor mais adequado com base na localização geográfica da cidade pesquisada: o IPMA é utilizado para Portugal, o AEMET para Espanha, o CWA para Taiwan e o Open-Meteo para os quinze restantes países suportados.

A arquitetura do *backend* segue o padrão *Provider Adapter* (Escrich, 2023): cada fornecedor é encapsulado num módulo independente que implementa a lógica de comunicação, autenticação e transformação de dados específica desse fornecedor. O núcleo da aplicação apenas conhece a interface comum, não os detalhes de cada API externa. Esta separação garante que a adição de um novo fornecedor, por exemplo,

uma agência meteorológica alemã ou brasileira exige apenas a criação de um novo módulo adaptador, sem qualquer alteração no código existente.

Importa delimitar o âmbito do presente trabalho. A seleção do fornecedor é realizada no *frontend*, não através de um *endpoint* unificado no *backend*, funcionalidade identificada como trabalho futuro. O sistema não implementa mecanismos de *caching* de respostas, autenticação de utilizadores, nem suporte a notificações ou alertas meteorológicos automáticos. A cobertura geográfica está igualmente circunscrita aos países suportados pelos quatro fornecedores integrados, não sendo garantida cobertura para regiões fora deste conjunto. Estas limitações não comprometem os objetivos definidos para o projeto, mas constituem direções naturais para desenvolvimentos futuros.

1.4. Estrutura do Relatório

O presente relatório encontra-se organizado em seis secções, cuja articulação segue a progressão lógica do trabalho desenvolvido, da contextualização teórica, passando pelas decisões de arquitetura, até à validação e conclusões.

Capítulo 1 - Introdução. O presente capítulo contextualiza o tema, justifica a escolha do problema, enuncia os objetivos do projeto e descreve a estrutura do documento.

Capítulo 2 - Estado da Arte. Este capítulo apresenta uma revisão da literatura relevante nos domínios da previsão meteorológica computacional, das APIs de dados abertos e dos padrões arquiteturais para integração de múltiplos fornecedores. São analisados os modelos globais de previsão (ECMWF, GFS, Open-Meteo) e os modelos locais (HIRLAM, ALADIN), bem como as interfaces de dados abertos disponibilizadas por agências nacionais. São ainda abordados os padrões de software relevantes, nomeadamente o *Adapter Pattern*, o *Aggregator Pattern* e a arquitetura REST.

Capítulo 3 - Análise e Requisitos. Este capítulo descreve o levantamento e análise de requisitos do sistema, incluindo os requisitos funcionais (tipos de previsão, fornecedores suportados, funcionalidades da interface) e não-funcionais (desempenho, extensibilidade, documentação automática). São também apresentados os casos de uso principais e as restrições identificadas durante a análise das APIs dos fornecedores, por exemplo, a ausência de condições atuais no AEMET, o limite de três dias do IPMA, ou a codificação ISO8859-1 das respostas do AEMET.

Capítulo 4 - Arquitetura e Implementação. Este capítulo detalha as decisões de arquitetura e a implementação técnica do sistema. Descreve a estrutura de módulos

do *backend* (*providers*, *routers*, *models*, *geocoding*), as escolhas tecnológicas (FastAPI, Pydantic, httpx, React) e os principais desafios de implementação encontrados, nomeadamente a normalização de modelos de dados heterogéneos, o tratamento do fluxo de dois passos da API AEMET, a codificação **ISO-8859-1** das respostas AEMET e o mapeamento de *datasets* por município no CWA.

Capítulo 5 -Testes e Validação. Este capítulo apresenta a estratégia de testes adotada, que compreende testes unitários sobre os módulos de *parsing* e conversão de unidades, testes de integração com *mocking* das APIs externas, e testes de contrato para deteção de alterações silenciosas nos contratos das APIs.

Capítulo 6 - Conclusões e Trabalho Futuro. O capítulo final sintetiza os resultados alcançados, avalia o grau de consecução dos objetivos inicialmente propostos e identifica as limitações atuais do sistema. São ainda apontadas direções para trabalho futuro, nomeadamente a implementação de um *endpoint* unificado com seleção automática de fornecedor no *backend*, a adição de novas agências nacionais e a introdução de uma camada de *caching* para reduzir a latência e proteger as chaves de API face a volumes elevados de utilização.

2. Fundamentação Teórica

O presente capítulo apresenta a revisão da literatura e o enquadramento conceptual que suporta as decisões de arquitetura e implementação do sistema desenvolvido. A exposição está organizada em quatro domínios complementares: a previsão meteorológica computacional e os modelos numéricos que a sustentam; o movimento de dados abertos nas agências meteorológicas nacionais e os serviços de API que dele resultam; os padrões arquiteturais de *software* utilizados na conceção do sistema; e as tecnologias de implementação selecionadas.

2.1. Previsão Meteorológica Numérica

A previsão meteorológica moderna assenta nos denominados modelos de Previsão Numérica do Tempo (*Numerical Weather Prediction*, NWP), que simulam o comportamento da atmosfera terrestre através da resolução numérica de equações diferenciais que descrevem a dinâmica dos fluidos atmosféricos, a transferência de radiação, a convecção e os processos de superfície. A capacidade de previsão destes modelos resulta da combinação entre a qualidade dos dados de inicialização obtidos por assimilação de observações provenientes de satélites, radiossondas, aeronaves e estações de superfície, e a resolução espacial e temporal com que a atmosfera é discretizada na grelha de cálculo. (Bi et al., 2023b)

2.1.1. Modelos Globais

Os modelos globais operam sobre todo o planeta e produzem previsões para horizontes de vários dias. O sistema mais amplamente reconhecido é o *Integrated Forecasting System* (IFS) do Centro Europeu de Previsão do Tempo a Médio Prazo (ECMWF), considerado na literatura o sistema de previsão global de maior exatidão disponível. O componente de alta resolução do IFS, o HRES, opera a uma resolução horizontal de $0,1^\circ$, equivalente a cerca de 9 km, com 137 níveis verticais, produzindo previsões até 10 dias com atualizações a cada seis horas. O sistema utiliza assimilação de dados 4D-Var (Assimilação variacional 4-dimensional), ingerindo cerca de 60 milhões de observações por ciclo de assimilação (Bauer et al., 2015; Frnda et al., 2022; Hersbach et al., 2020; Rabier et al., 2000). Em junho de 2023, o ciclo 48r1 do IFS igualou a resolução do ensemble (ENS, 51 membros) à do HRES, representando um avanço significativo na capacidade de quantificação da incerteza das previsões (Mass & Vuckovic, 2023).

O *Global Forecast System* (GFS), operado pelo *National Centers for Environmental Prediction* (NCEP) dos Estados Unidos, constitui a alternativa de referência no contexto norte-americano, operando a uma resolução horizontal de aproximadamente 13 km com 127 níveis verticais. O ensemble associado, o GEFS (versão 12), incorpora 31 membros baseados no núcleo dinâmico *Finite-Volume Cubed-Sphere* (FV3), com resolução de grade de $0,25^\circ$ e 64 níveis verticais. Ambos os sistemas, IFS e GFS, são utilizados como condições de fronteira para modelos regionais de maior resolução operados pelas agências meteorológicas nacionais (Zhou et al., 2022).

2.1.2. Modelos Regionais e Locais

A principal limitação dos modelos globais reside na sua resolução espacial: a fenomenologia meteorológica de escala local incluindo a influência da orografia, dos contrastes terra-mar, das linhas costeiras e da cobertura do solo não é adequadamente representada a resoluções superiores a 5–10 km. Os modelos regionais ou mesoscala abordam esta limitação através do chamado *downscaling* dinâmico (Giorgi, 2019; Prein et al., 2015): utilizam as previsões de um modelo global como condições de fronteira e resolvem a dinâmica atmosférica a resoluções significativamente mais finas, tipicamente entre 1 e 5 km. Estudos comparativos demonstram que a precisão dos modelos melhora consistentemente com o aumento

da resolução, sendo o impacto particularmente pronunciado em regiões costeiras, montanhosas e insulares (Frnda et al., 2022).

Entre os modelos regionais mais utilizados na Europa contam-se o *Application of Research to Operations at Mesoscale* (AROME), operado pelo serviço meteorológico francês *Météo-France* com resolução de 1,3 km e adotado por vários países europeus (Seity et al., 2011), e o ALADIN, modelo regional de previsão desenvolvido em cooperação internacional e utilizado pela AEMET espanhola, entre outros (Bubnová et al., 1995). O *High Resolution Limited Area Model* (HIRLAM), desenvolvido em consórcio pelos serviços meteorológicos nórdicos, opera tipicamente a resoluções de 2 a 3 km e constitui a base dos modelos operacionais de países como Suécia, Noruega e Países Baixos (Bengtsson et al., 2017).

Em Taiwan, a *Central Weather Administration* (CWA) opera o TWRF (*Typhoon WRF*), um sistema regional baseado no modelo ARW-WRF com configuração de duplo domínio aninhado a 15 km e 3 km de resolução, dedicado à previsão de tufões e fenómenos de mesoscala na Ásia Oriental (Hsiao et al., 2020). A necessidade de modelos especializados nesta região é ilustrada pelo facto de os modelos globais, embora competentes na previsão de sistemas sinópticos, apresentarem desempenho variável para sistemas que envolvem múltiplas escalas, como os tufões que afetam Taiwan e o litoral do Pacífico Ocidental. (Emanuel & Zhang, 2016)

2.2. A Inteligência Artificial em Previsão Meteorológica

A última meia década assistiu ao surgimento de modelos de aprendizagem automática que rivalizam e, em certas métricas, superam os sistemas NWP físicos clássicos. O *Pangu-Weather*, desenvolvido pela Huawei e publicado na *Nature* em 2023, demonstrou que uma rede neuronal tridimensional baseada em *transformers* pode produzir previsões de médio prazo com precisão comparável ao HRES do ECMWF, a uma fração do custo computacional (Bi et al., 2023b). O *GraphCast*, publicado pela Google *DeepMind* na *Science* no mesmo ano, utilizou redes neurais sobre grafos treinadas em 39 anos de reanálise ERA5 para atingir desempenho superior ao HRES em 90% das variáveis e horizontes de previsão avaliados (Lam et al., 2023).

O ECMWF respondeu com o seu próprio sistema de inteligência artificial (IA), o *Artificial Intelligence Forecasting System* (AIFS), que se tornou operacional em fevereiro de 2025 como o primeiro sistema de previsão global baseado em IA a funcionar em modo operacional numa agência meteorológica de referência (European

Centre for Medium-Range Weather Forecasts (ECMWF), 2025; Lam et al., 2023). O AIFS gera previsões globais a 10 dias em minutos numa GPU, com uma eficiência energética estimada cerca de 1.000 vezes superior aos sistemas físicos convencionais, e o seu ensemble de 51 membros (AIFS-ENS) tornou-se operacional em julho de 2025 (European Centre for Medium-Range Weather Forecasts (ECMWF), 2025).

Estes desenvolvimentos são relevantes para o contexto do presente projeto porque alteram o panorama de fornecedores de dados meteorológicos a médio prazo à medida que modelos de IA globais se tornem mais acessíveis, a vantagem relativa dos modelos regionais nacionais poderá ser parcialmente erodida, ou os próprios modelos regionais poderão adotar arquiteturas de aprendizagem automática para aperfeiçoar as suas previsões de alta resolução (Lam et al., 2023).

2.3. Dados Abertos Meteorológicos e APIs Nacionais

2.3.1. O Movimento de Dados Abertos em Meteorologia

A disseminação de dados meteorológicos através de interfaces programáticas abertas é relativamente recente enquanto prática sistemática. Durante décadas, o acesso a dados de previsão de alta qualidade esteve condicionado a acordos institucionais entre os serviços meteorológicos nacionais e os centros mundiais de previsão do tempo, sob o quadro do Sistema de Informação da Organização Meteorológica Mundial (WMO-IS). A abertura progressiva destes dados ao público e a programadores externos começou a ganhar expressão na segunda década do século XXI, impulsionada por políticas de dados abertos nos sectores público e privado e pela crescente valorização económica dos dados meteorológicos em aplicações como a agricultura de precisão, a mobilidade autónoma e a gestão de energias renováveis (Adrian et al., 2021).

Um marco significativo foi o anúncio do ECMWF, em 2023, de uma transição progressiva para uma política de dados abertos, disponibilizando centenas de produtos de previsão a qualquer utilizador. A WMO enquadrou esta tendência no âmbito do seu programa WIS 2.0 (WMO Information System 2.0), que adotou arquitetura *web*, padrões abertos e princípios RESTful como sucessor do sistema legado baseado em SOA e XML. O WIS 2.0 prevê que 90% dos membros da WMO completem a transição até 2030, assegurando partilha em tempo real de dados meteorológicos, hidrológicos e climáticos através de catálogos de descoberta compatíveis com a API OGC. (Frnda et al., 2022)

2.3.2. Os Fornecedores Integrados no Presente Sistema

O sistema desenvolvido integra quatro fornecedores de dados meteorológicos, selecionados em função da cobertura geográfica, da disponibilidade de API pública documentada e da qualidade dos dados fornecidos.

A O **Open-Meteo** é um serviço de API meteorológica de código aberto, de acesso gratuito, que agrega dados de previsão provenientes de vários serviços meteorológicos nacionais e os disponibiliza numa interface JSON unificada. Para cada localização, o Open-Meteo seleciona automaticamente o modelo de maior resolução aplicável à área geográfica em questão, integrando modelos como o ECMWF IFS, o GFS, o AROME, o ICON e o ERA5, para dados históricos. O serviço não requer chave de API para utilização não comercial (até 10.000 chamadas diárias) e opera sob a licença de dados CC BY 4.0. (Zippenfenig, 2024)

O **IPMA** (Instituto Português do Mar e da Atmosfera) é a autoridade meteorológica nacional de Portugal, responsável pela emissão de avisos, pela monitorização do clima e pela operação de modelos de previsão regional. O IPMA disponibiliza uma API de dados abertos que expõe previsões de até cinco dias para um conjunto de localidades portuguesas, incluindo o arquipélago dos Açores e da Madeira. O acesso é gratuito e não requer autenticação, e as previsões baseiam-se nos modelos regionais operados pelo instituto, calibrados especificamente para o território português. (Instituto Português do Mar e da Atmosfera (IPMA), 2024b)

A **AEMET** (*Agencia Estatal de Meteorología*) é o serviço meteorológico estatal de Espanha. A sua plataforma OpenData expõe uma API REST que permite aceder a previsões diárias até sete dias e horárias até 48 horas para todos os municípios espanhóis identificados pelo código INE (Instituto Nacional de Estadística). O registo é gratuito, mas requer chave de API. As respostas são codificadas em ISO-8859-1 (latin-1), refletindo a herança histórica do sistema, e o acesso segue um fluxo de dois passos: um primeiro pedido devolve um URL temporário que deve ser acedido num segundo pedido para obter o *payload* efetivo. (Agencia Estatal de Meteorología (AEMET), 2024)

A **CWA** (*Central Weather Administration*) de Taiwan opera uma plataforma de dados abertos que disponibiliza dados de observação em tempo real, bem como previsões de 36 horas e previsões diárias há dois dias. O acesso é gratuito mediante registo e obtenção de chave de API. Os dados de previsão estão organizados por *datasets* individuais por condado, sendo necessário conhecer o identificador específico

de cada um (ex.: F-D0047-061 para Taipei). (Central Weather Administration (CWA), 2024)

A heterogeneidade entre estes quatro fornecedores em termos de protocolos de autenticação, formatos de resposta JSON, codificações de texto, organização geográfica e disponibilidade de tipos de previsão constitui precisamente o problema que o presente sistema se propõe resolver, e justifica a adoção de uma arquitetura baseada no padrão *Adapter* (Escrich, 2023).

2.4. Padrões de Arquitetura de Software

O padrão *Adapter* é um dos 23 padrões de *design* estruturais catalogados por Gamma, Helm, Johnson e Vlissides na obra seminal *Design Patterns: Elements of Reusable Object-Oriented Software*, publicada em 1994. Segundo a sua definição original, o padrão *Adapter* converte a interface de uma classe noutra interface que o cliente espera encontrar, permitindo que classes com interfaces incompatíveis trabalhem em conjunto. O padrão é estruturalmente composto por quatro participantes: o *Target* (interface esperada pelo cliente), o *Adaptee* (classe existente com interface incompatível), o *Adapter* (envolve o *Adaptee* e implementa o *Target*), e o *Client* (utiliza objetos conformes com o *Target*). (Gamma et al., 1994)

A aplicabilidade do padrão é particularmente relevante em dois contextos: quando se pretende utilizar uma classe existente cuja interface não corresponde à esperada, e quando se pretende criar uma classe reutilizável que coopere com classes não relacionadas ou imprevisíveis. No contexto da integração de APIs de terceiros, o padrão *Adapter* é amplamente utilizado para criar uma camada de abstração que isola o código cliente da especificidade de cada fornecedor, permitindo substituir ou adicionar fornecedores sem alterar o núcleo da aplicação. Ao criar esta camada de abstração entre o código de negócio e as interfaces externas, torna o sistema mais resiliente a alterações nos contratos das APIs de terceiros e facilita a testabilidade através da substituição de adaptadores reais por *mocks* nos testes automatizados. (Escrich, 2023)

Três décadas após a sua publicação, os padrões GoF (*Gang of Four*) mantêm relevância plena no desenvolvimento de *software* moderno, incluindo em contextos de micro-serviços, integração de APIs e desenvolvimento em Python e JavaScript. No presente projeto, cada fornecedor meteorológico é encapsulado num módulo adaptador independente (ex.: `app/providers/aemet/adapter.py`), que implementa a lógica de comunicação HTTP, autenticação, descodificação de respostas e transformação dos dados para os modelos internos. Os *routers* FastAPI apenas

interagem com estes módulos através de funções com assinaturas tipificadas, desconhecendo os detalhes de cada API externa. (Hussain et al., 2017)

2.4.1. O padrão *Aggregator* e API Gateway

Em arquiteturas de micros-serviços, o padrão *Aggregator* descreve um serviço que recebe pedidos de clientes, invoca múltiplos serviços internos, e combina as suas respostas num resultado unificado devolvido ao cliente (TatvaSoft, 2023). O *Aggregator* é frequentemente implementado em conjunto com o padrão API Gateway, que atua como ponto de entrada único para um conjunto de serviços internos, realizando funções transversais como autenticação, *rate limiting*, *routing* e transformação de respostas (IASA Global, 2023).

A motivação principal para este padrão é a simplificação da perspetiva do cliente: em vez de interagir com múltiplos serviços heterogéneos, o cliente interage com uma interface unificada que oculta a complexidade do sistema subjacente. Esta propriedade alinha-se diretamente com o objetivo do presente projeto: o utilizador final da interface *web* não precisa de saber que Portugal é servido pelo IPMA, Espanha pela AEMET e Taiwan pela CWA, a lógica de seleção do fornecedor mais adequado é gerida internamente pelo sistema. (IASA Global, 2023)

2.4.2. Arquitetura REST

O estilo arquitetural REST foi formalizado por Roy Fielding na sua dissertação de doutoramento na Universidade da Califórnia, em 2000 (Fielding, 2000). REST define um conjunto de seis restrições que, aplicadas conjuntamente, produzem sistemas com as propriedades desejadas de desempenho, escalabilidade, simplicidade, modificabilidade e fiabilidade:

1. **Cliente-Servidor:** separação de preocupações entre a interface do utilizador e o armazenamento de dados, permitindo a evolução independente de ambos os componentes;
2. **Sem estado (*Stateless*):** cada pedido do cliente deve conter toda a informação necessária para o servidor o processar, sem dependência de contexto de pedidos anteriores;
3. **Cacheável:** as respostas devem indicar se podem ou não ser armazenadas em cache, de forma a reduzir interações desnecessárias com o servidor;
4. **Sistema em camadas (*Layered System*):** o cliente não precisa de saber se está a comunicar diretamente com o servidor ou com um intermediário;

5. **Interface uniforme:** a utilização de URIs para identificar recursos, verbos HTTP com semântica bem definida e mensagens autodescritivas;
6. **Código a pedido (*Code on Demand*, opcional):** possibilidade de o servidor transferir lógica executável para o cliente.

O modelo de maturidade de Richardson, proposto em 2008, classifica APIs *Web* em quatro níveis de conformidade com os princípios REST. O nível 0 utiliza um único *endpoint* com HTTP apenas como transporte; o nível 1 introduz recursos identificados por URIs distintos; o nível 2 acrescenta a utilização correta dos verbos HTTP e códigos de estado; o nível 3 implementa *Hypermedia as the Engine of Application State* (HATEOS), considerado o nível verdadeiramente RESTful segundo a definição de Fielding (Fielding, 2000; TatvaSoft, 2023). Um estudo empírico em larga escala sobre tráfego real de APIs confirma que a maioria dos fornecedores de API se situa precisamente no nível 2, com boa adoção dos verbos HTTP e códigos de estado mas pouca adoção de hipermédia (Rodríguez et al., 2016). Na prática, a maioria das APIs públicas, incluindo as integradas no presente projeto, alinha-se com o nível 2: recursos bem definidos, método GET para consulta, e códigos de estado HTTP (200, 404, 401, 502) para comunicar o resultado das operações.

2.5. Tecnologias de Implementação

2.5.1. FastAPI

O FastAPI é uma *framework* Python moderna para o desenvolvimento de APIs *web*, introduzida em 2018 por Sebastián Ramírez e construída sobre o Starlette (toolkit ASGI) e o Pydantic (validação de dados) (Ramírez, 2024). O seu diferencial central relativamente a frameworks anteriores como Flask ou Django REST *Framework* reside em três aspetos: o suporte nativo a programação assíncrona via *async/await*, a geração automática de documentação OpenAPI/Swagger a partir de anotações de tipo Python, e a validação automática de dados de entrada e saída através de modelos Pydantic.

A distinção entre *Web Server Gateway Interface* (WSGI) e *Asynchronous Server Gateway Interface* (ASGI) é fundamental para compreender a vantagem do FastAPI em contextos de integração de APIs externas. Os *frameworks* WSGI, como o Flask, processam pedidos de forma síncrona cada pedido bloqueia o processo até receber resposta. Em aplicações que dependem de múltiplas chamadas HTTP a serviços externos, este modelo implica que as chamadas ocorrem sequencialmente, multiplicando a latência. O FastAPI, construído sobre ASGI e o servidor Uvicorn,

suporta I/O não bloqueante: múltiplas chamadas HTTP externas podem ser iniciadas em paralelo com `asyncio.gather()`, reduzindo a latência total para o máximo das latências individuais em vez da sua soma. O Pydantic, componente central do FastAPI, utiliza desde a versão 2.0 um núcleo em Rust para a validação de dados, resultando em ganhos de desempenho significativos e garantindo que todos os dados que entram e saem da API respeitam os esquemas definidos, com geração automática de erros descritivos em caso de violação.

2.5.2. HTTPX e Comunicação Assíncrona

A comunicação com as APIs externas dos fornecedores meteorológicos é realizada através da biblioteca HTTPX, um cliente HTTP de nova geração para Python 3 que suporta tanto interfaces síncronas como assíncronas, bem como HTTP/1.1 e HTTP/2 (Christie & Encode OSS Ltd., 2024). Ao contrário da biblioteca *requests*, amplamente utilizada em Python síncrono, o *httpx* suporta clientes assíncronos (*AsyncClient*) que permitem iniciar múltiplos pedidos HTTP em paralelo dentro do mesmo ciclo de eventos, sem criar *threads* adicionais. A biblioteca é a escolha recomendada pelo próprio FastAPI para chamadas HTTP externas em contextos assíncronos, dado que a sua *AsyncClient* partilha o mesmo modelo de concorrência *asyncio*. Esta propriedade é particularmente relevante para o presente sistema, no qual a interface web pode desencadear chamadas simultâneas a diferentes *endpoints* de um mesmo fornecedor que são resolvidas em paralelo com ganhos substanciais de latência.

2.5.3. React

O React é uma biblioteca JavaScript de código aberto, desenvolvida pela Meta (anteriormente *Facebook*) e lançada publicamente em 2013, destinada à construção de interfaces de utilizador baseadas em componentes reutilizáveis (Meta Platforms, 2024). A sua arquitetura centra-se em dois conceitos fundamentais: a componentização (a interface é decomposta em componentes independentes com o seu próprio estado e ciclo de vida) e a renderização declarativa (o programador descreve como a interface deve parecer para um dado estado, e o React encarrega-se de atualizar eficientemente o *Document Object Model* (DOM) quando o estado muda).

O React introduziu o conceito de DOM virtual (Virtual DOM) que em vez de manipular diretamente o DOM do navegador a cada atualização, operação computacionalmente cara, o React mantém uma representação em memória da árvore de componentes e calcula o conjunto mínimo de alterações necessárias antes de as aplicar ao DOM real, otimizando o desempenho de renderização (Meta Platforms,

2024). Com a introdução dos *Hooks* (versão 16.8, 2019), o React tornou possível gerir estado e efeitos secundários em componentes funcionais, eliminando a necessidade de classes e simplificando substancialmente o modelo mental de desenvolvimento (W3Schools, 2026).

No presente projeto, o *Context* API do React é utilizado para gerir preferências globais do utilizador (unidades de temperatura e velocidade do vento) que precisam de estar acessíveis em múltiplos componentes sem necessidade de passagem de *props* por toda a hierarquia de componentes (*prop drilling*). O *Context* API constitui a solução nativa do React para este tipo de estado partilhado de âmbito global, mas de baixa frequência de atualização, sendo preferível a bibliotecas de gestão de estado de terceiros como *Redux* em aplicações de complexidade moderada. Os *hooks* personalizados (*useWeather*, *useGeolocation*, *useAllLocations*) encapsulam a lógica de comunicação com o *backend*, separando claramente a camada de obtenção de dados da camada de apresentação.

2.5.4. Geocodificação e APIs de Geolocalização

A geocodificação, processo de converter um nome de localidade em coordenadas geográficas (latitude e longitude), é um componente transversal do sistema, utilizado tanto para a pesquisa de cidades no *frontend* como para a resolução de nomes no *backend* antes de chamar a API do Open-Meteo. O sistema utiliza dois serviços distintos para este efeito: a API de geocodificação do Open-Meteo para a resolução direta de nomes de cidades em coordenadas, e o Nominatim da OpenStreetMap para a geocodificação inversa (conversão de coordenadas GPS em nome de localidade), utilizada pela funcionalidade de geolocalização automática. (Zippenfenig, 2024)

A geocodificação inversa via Nominatim é exposta pelo *backend* através de um *endpoint proxy* (GET */geo/reverse*), evitando que o *frontend* faça chamadas diretas a um domínio externo e contorne assim as restrições *CrossOrigin Resource Sharing* (CORS) impostas pelos navegadores modernos. Esta é uma solução arquitetural comum em sistemas *full-stack*: o *backend* atua como intermediário para serviços externos que não implementam os cabeçalhos CORS necessários para consumo direto pelo *browser* (TatvaSoft, 2023).

2.5.5. OpenAPI e Documentação Automática

A especificação OpenAPI (anteriormente conhecida como *Swagger*) é um padrão aberto para a descrição de interfaces de APIs HTTP RESTful, alinhado com os princípios REST definidos por Fielding (Fielding, 2000). Uma especificação OpenAPI

descreve de forma estruturada e legível por máquina os *endpoints* disponíveis, os seus parâmetros, os modelos de dados de entrada e saída, os mecanismos de autenticação e os códigos de resposta. O FastAPI gera automaticamente uma especificação OpenAPI a partir das anotações de tipo e dos modelos Pydantic definidos no código, sem necessidade de manutenção manual (Ramírez, 2024). Esta especificação é servida em `/openapi.json` e é utilizada para renderizar a interface interativa Swagger UI em `/docs` e a interface ReDoc em `/redoc`, que permitem explorar e testar todos os *endpoints* da API diretamente no navegador. A documentação automática é particularmente valiosa em sistemas de integração como o presente, onde a clareza das interfaces entre *backend* e *frontend* é crítica e a documentação manual tende a desatualizar-se rapidamente.

3. Metodologia

O desenvolvimento do presente projeto seguiu uma abordagem iterativa e incremental, estruturada em duas fases sequenciais principais: *backend* e *frontend*; com uma duração total aproximada de três meses e meio. O relatório foi sendo redigido em paralelo ao longo de todo o período de desenvolvimento, à medida que cada componente era concluído.

3.1. Planeamento e Organização

A organização do projeto foi suportada pela ferramenta de diagramação colaborativa *tlldraw*, utilizada para esquematizar e registar iterativamente as ideias em desenvolvimento. Nesta ferramenta foram elaborados os requisitos funcionais e não-funcionais do sistema, diagramas de arquitetura que ilustram a relação entre os subsistemas, e diagramas de casos de uso que descrevem o comportamento esperado da aplicação do ponto de vista do utilizador. Esta abordagem visual permitiu validar decisões de design antes da sua implementação e manter uma visão global do sistema à medida que a complexidade aumentava.

O trabalho foi dividido em três fases com durações aproximadas:

1. **Fase 1 - Backend (aproximadamente 2,5 meses):** concepção da arquitetura, integração dos fornecedores meteorológicos, implementação dos *endpoints* REST;
2. **Fase 2 - Frontend (aproximadamente 1/2 mês):** desenvolvimento da interface React, integração com o *backend*, implementação das funcionalidades de pesquisa, geolocalização, histórico de pesquisas e visualização gráfica.

3. **Fase 3 – Testes (aproximadamente ½ mês):** realização de testes e verificação do comportamento adequado da aplicação, e correção das falhas identificadas.

3.1.1. Seleção dos Fornecedores de Dados

A seleção dos fornecedores meteorológicos a integrar resultou de um processo de investigação documental. O IPMA foi o ponto de partida natural, por ser a agência meteorológica nacional do país de residência do autor e o serviço de referência para a consulta quotidiana de previsões. Os restantes fornecedores: AEMET, CWA e Open-Meteo; foram identificados após pesquisa alargada sobre APIs meteorológicas com dados abertos, privilegiando-se fontes que reunissem três critérios:

- disponibilização de dados através de uma API pública documentada
- cobertura geográfica relevante
- uma estrutura de resposta suficientemente acessível para permitir integração num prazo razoável.

Para cada fornecedor selecionado, o processo de integração iniciou-se com uma fase de análise da documentação oficial e de exploração prática da API verificando quais os dados disponíveis, o formato das respostas JSON e os requisitos de autenticação antes de qualquer implementação. Esta análise prévia revelou-se determinante para antecipar as heterogeneidades entre fornecedores, nomeadamente a codificação ISO-8859-1 das respostas do AEMET, o fluxo de dois passos da mesma API, ou a ausência de dados de condições atuais no IPMA e no AEMET.

3.1.2. Desenvolvimento e Controlo de Versões

O desenvolvimento foi realizado integralmente em ambiente local, utilizando o editor de texto Neovim (*Neovim*, s.d.), o sistema de controlo de versões Git (*Git*, s.d.) para registo incremental das alterações ao código, e um ambiente virtual Python (venv) para isolamento das dependências do *backend* (*Venv — Creation of Virtual Environments*, s.d.). O *frontend* foi desenvolvido em React com recurso ao conjunto de ferramentas padrão da plataforma (*Create React App*). A aplicação foi validada exclusivamente em ambiente local, não tendo sido realizado *deployment* em infraestrutura de produção no âmbito deste projeto.

3.1.3. Estratégia de Testes

Os testes automatizados foram escritos numa fase posterior à implementação das funcionalidades, seguindo uma lógica de validação do comportamento real do sistema. Este processo implicou, em alguns casos, a refatorização do código existente

para melhorar a testabilidade, nomeadamente o isolamento das chamadas HTTP externas em módulos adaptadores dedicados, o que permitiu a sua substituição por *mocks* nos testes sem alterar a lógica de negócio. A *suite* de testes resultante cobre três camadas: testes unitários sobre os módulos de *parsing* e resolução de cidades, testes de integração sobre os *endpoints* REST com simulação das respostas externas e verificações de contrato destinadas a detetar alterações silenciosas nos formatos de resposta das APIs dos fornecedores.

4. Análise e Requisitos

O presente capítulo descreve o processo de levantamento e análise de requisitos do sistema ***Local Weather***, identificando as funcionalidades que o sistema deve oferecer (**requisitos funcionais**), as propriedades de qualidade que deve respeitar (**requisitos não-funcionais**), os cenários de utilização previstos (**casos de uso**) e as restrições impostas pelas APIs dos fornecedores externos.

4.1. Requisitos Funcionais

Os requisitos funcionais definem o comportamento que o sistema deve apresentar em resposta a entradas ou em determinadas condições. São apresentados separadamente para o *backend* e para o *frontend*, refletindo a arquitetura de dois subsistemas independentes.

4.1.1. Backend

A Tabela 1 apresenta os requisitos funcionais identificados para o *backend*, organizados por fornecedor de dados meteorológicos e cobrindo os *endpoints* de previsão, pesquisa de localidades e geocodificação.

Tabela 1 - Requisitos funcionais do *backend*

Código	Descrição
RF01	O sistema deve expor um <i>endpoint</i> REST para obter as condições meteorológicas atuais para qualquer cidade com cobertura pelo Open-Meteo, identificada por nome e país opcional
RF02	O sistema deve expor um <i>endpoint</i> REST para obter a previsão horária Open-Meteo para uma cidade, com horizonte configurável de 1 a 16 dias.
RF03	O sistema deve expor um <i>endpoint</i> REST para obter a previsão diária

	Open-Meteo para uma cidade, com horizonte configurável de 1 a 16 dias.
RF04	O sistema deve expor um <i>endpoint</i> REST para pesquisar municípios espanhóis por nome, retornando o código INE correspondente.
RF05	O sistema deve expor um <i>endpoint</i> REST para obter a previsão diária AEMET para um município espanhol, com horizonte de 7 dias, identificado pelo código INE.
RF06	O sistema deve expor um <i>endpoint</i> REST para obter a previsão horária AEMET para um município espanhol, com horizonte de 48 horas, identificado pelo código INE.
RF07	O sistema deve expor um <i>endpoint</i> REST para listar todas as cidades portuguesas suportadas pelo IPMA.
RF08	O sistema deve expor um <i>endpoint</i> REST para obter a previsão diária IPMA para uma cidade portuguesa, com horizonte de 5 dias.
RF09	O sistema deve expor um <i>endpoint</i> REST para listar todas as localizações de Taiwan suportadas pela CWA (22 condados e cidades).
RF10	O sistema deve expor um <i>endpoint</i> REST para obter as observações meteorológicas atuais nas estações da CWA para uma localização de Taiwan.
RF11	O sistema deve expor um <i>endpoint</i> REST para obter a previsão CWA de 36 horas (3 períodos de 12 horas) para uma localização de Taiwan.
RF12	O sistema deve expor um <i>endpoint</i> REST para obter a previsão diária CWA a 7 dias para uma localização de Taiwan.
RF13	O sistema deve expor um <i>endpoint</i> REST de geocodificação inversa que converta coordenadas GPS (latitude, longitude) no nome da cidade e país correspondentes
RF14	O sistema deve expor um <i>endpoint</i> de verificação de estado

	(/health) que informe o estado operacional de cada fornecedor integrado.
RF15	O sistema deve resolver automaticamente o nome de uma cidade em coordenadas geográficas antes de consultar a API do Open-Meteo, suportando uma dica de país opcional para desambiguação
RF16	O sistema deve gerar e disponibilizar documentação interativa da API em conformidade com a especificação OpenAPI, acessível via Swagger UI e ReDoc

4.1.2. Frontend

A Tabela 2 apresenta os requisitos funcionais identificados para a interface web, relativos à pesquisa de cidades, à apresentação dos dados meteorológicos e à personalização da experiência do utilizador.

Tabela 2 - Requisitos Funcionais do *Frontend*

Código	Descrição
RF17	A interface deve permitir ao utilizador pesquisar uma cidade por nome, apresentando resultados de todos os fornecedores em tempo real numa lista unificada com bandeira, nome e país.
RF18	A interface deve seleccionar automaticamente o fornecedor mais adequado com base no país da cidade seleccionada: IPMA para Portugal, AEMET para Espanha, CWA para Taiwan, e Open-Meteo para os restantes países suportados
RF19	A interface deve apresentar as condições meteorológicas atuais (temperatura, humidade, vento, pressão, precipitação) quando o fornecedor seleccionado as disponibilize.
RF20	A interface deve apresentar a previsão horária em colunas com <i>scroll</i> horizontal, incluindo temperatura, probabilidade de precipitação e vento, quando o fornecedor a disponibilize.
RF21	A interface deve apresentar a previsão diária em linhas com temperaturas máxima e mínima, descrição e probabilidade de precipitação, quando o fornecedor a disponibilize.
RF22	A interface deve visualizar graficamente a previsão de temperatura horária (curva de linha com área sombreada) e diária (gráfico de barras com máximas e mínimas).

RF23	A interface deve disponibilizar um botão de geolocalização automática que detete a posição GPS do utilizador, resolva as coordenadas para um nome de cidade via /geo/reverse, e apresente a previsão correspondente.
RF24	A interface deve manter um histórico das últimas 8 pesquisas do utilizador, persistido em <code>localStorage</code> , com possibilidade de repetir, remover individualmente ou limpar o histórico completo.
RF25	A interface deve permitir ao utilizador alternar as unidades de temperatura entre graus Celsius (°C) e Fahrenheit (°F), e as unidades de velocidade do vento entre km/h e mph, com persistência da preferência em <code>localStorage</code> .
RF26	A interface deve suportar dois temas visuais, claro e escuro, seleccionáveis pelo utilizador através de um botão de alternância no cabeçalho.
RF27	A interface deve suportar dois idiomas, inglês e português, seleccionáveis pelo utilizador, com persistência da preferência em <code>localStorage</code> .
RF28	A interface deve incluir uma página “About” com a motivação do projeto, a cobertura geográfica por fornecedor, e a referência ao repositório público no GitHub.
RF29	A interface deve apresentar uma nota informativa por pesquisa indicando qual o fornecedor ativo, a cobertura geográfica, e os tipos de previsão disponíveis e não disponíveis

4.2. Requisitos Não-Funcionais

Os requisitos não-funcionais definem as propriedades de qualidade que o sistema deve respeitar, independentemente da funcionalidade específica. A Tabela 3 apresenta os dez requisitos identificados, organizados por categoria (desempenho, extensibilidade, segurança, usabilidade, entre outras).

Tabela 3 - Requisitos Não-Funcionais do Sistema

Código	Categoria	Descrição
RNF01	Desempenho	As chamadas a fornecedores independentes no mesmo pedido devem ser executadas em paralelo com <code>asyncio.gather()</code> , minimizando a latência total ao máximo das latências individuais.

RNF02	Extensibilidade	A arquitetura deve permitir a integração de um novo fornecedor meteorológico através da criação de um módulo adaptador isolado, sem alteração dos módulos existentes.
RNF03	Documentação	Todos os <i>endpoints</i> da API devem estar documentados de forma automática e interativa via OpenAPI/Swagger UI, sem necessidade de manutenção manual.
RNF04	Testabilidade	O código deve ser estruturado de forma a permitir a substituição das dependências externas (chamadas HTTP às APIs dos fornecedores) por <i>mocks</i> nos testes, sem alteração da lógica de negócio.
RNF05	Segurança	As chaves de API dos fornecedores AEMET e CWA devem ser carregadas exclusivamente a partir de variáveis de ambiente (.env), nunca incluídas no código fonte ou em repositórios de controlo de versões.
RNF06	Usabilidade	A seleção do fornecedor deve ser transparente para o utilizador final, a lógica de encaminhamento deve ser gerida pela aplicação sem intervenção do utilizador
RNF07	Compatibilidade	A API deve expor cabeçalhos CORS que permitam o seu consumo a partir de qualquer origem, suportando apenas o método HTTP GET (API de leitura).
RNF08	Isolamento	Cada fornecedor deve ser encapsulado num módulo independente, de modo que a alteração ou remoção de um fornecedor não afete os restantes módulos do sistema.
RNF09	Persistência de Preferências	As preferências do utilizador (tema, idioma, unidades de medida e histórico de pesquisas) devem ser persistidas localmente no localStorage do browser, sem necessidade de conta ou autenticação.
RNF10	Responsividade	A interface deve adaptar-se corretamente a diferentes tamanhos de ecrã, incluindo dispositivos

	móveis e ecrãs de secretária
--	------------------------------

4.3. Casos de Uso

São identificados cinco casos de uso principais, correspondendo aos cenários de interação mais relevantes entre o utilizador e o sistema.

4.3.1. CU01 - Pesquisar Cidade e Consultar Previsão

Tabela 4 - Caso de uso CU01 - Pesquisar Cidade e

Identificador	CU01
Nome	Pesquisar cidade e consultar previsão meteorológica
Ator	Utilizador
Pré-condições	O utilizador tem acesso à interface web e o backend está em funcionamento.
Fluxo principal	1. O utilizador escreve o nome de uma cidade na barra de pesquisa; 2. O sistema apresenta resultados em tempo real provenientes de todos os fornecedores disponíveis; 3. O utilizador seleciona uma cidade da lista de resultados; 4. O sistema determina o fornecedor adequado com base no país da cidade selecionada; 5. O sistema consulta o fornecedor e apresenta as condições atuais (se disponíveis), a previsão horária (se disponível) e a previsão diária; 6. O sistema guarda a pesquisa no histórico de pesquisas recentes.
Fluxos alternativos	4a. Se o país for Portugal, é utilizado o IPMA (previsão diária de 5 dias, sem condições atuais nem previsão horária). 4b. Se o país for Espanha, é utilizado o AEMET (previsão diária de 7 dias e horária de 48h, sem condições atuais). 4c. Se a localização for em Taiwan, é utilizado o CWA (condições atuais, 36h e 2 dias).
Pós condições	A previsão meteorológica é apresentada na interface e a pesquisa é registada no histórico
Requisitos	RF17, RF18, RF19, RF20, RF21, RF22, RF24, RF29

4.3.2. CU02 - Usar Geolocalização Automática

Tabela 5 - Caso de uso CU02 - Usar Geolocalização

Identificador	CU02
Nome	Obter previsão para a localização atual do utilizador
Ator	Utilizador
Pré-condições	O browser do utilizador suporta a API navigator.geolocation e o utilizador concede permissão de acesso à localização.
Fluxo principal	1. O utilizador escreve o nome de uma cidade na barra de pesquisa; 2. O sistema apresenta resultados em tempo real provenientes de todos os fornecedores disponíveis; 3. O utilizador seleciona uma cidade da lista de resultados; 4. O sistema determina o fornecedor adequado com base no país da cidade selecionada; 5. O sistema consulta o fornecedor e apresenta as condições atuais (se disponíveis), a previsão horária (se disponível) e a previsão diária; 6. O sistema guarda a pesquisa no histórico de pesquisas recentes.
Fluxos alternativos	4a. Se o país for Portugal, é utilizado o IPMA (previsão diária de 5 dias, sem condições atuais nem previsão horária). 4b. Se o país for Espanha, é utilizado o AEMET (previsão diária de 7 dias e horária de 48h, sem condições atuais). 4c. Se a localização for em Taiwan, é utilizado o CWA (condições atuais, 36h e 2 dias).
Pós condições	A previsão meteorológica é apresentada na interface e a pesquisa é registada no histórico
Requisitos	RF17, RF18, RF19, RF20, RF21, RF22, RF24, RF29

4.3.3. CU03 — Repetir Pesquisa Recente

Tabela 6 - Caso de uso CU03 - Repetir Pesquisa Recente

Identificador	CU03
Nome	Repetir uma pesquisa do histórico recente
Ator	Utilizador
Pré-	O utilizador efetuou pelo menos uma pesquisa anterior na aplicação

condições	no mesmo browser.
Fluxo principal	1. O sistema apresenta os chips de pesquisas recentes (máximo 8) abaixo da barra de pesquisa. 2. O utilizador clica num chip de pesquisa recente. 3. O sistema repete a consulta ao fornecedor correspondente e atualiza os dados apresentados. 4. A pesquisa é movida para o topo do histórico
Fluxos alternativos	2a. O utilizador clica no botão X de um chip para o remover do histórico. 2b. O utilizador clica em “Limpar tudo” para apagar o histórico completo.
Pós condições	A previsão atualizada é apresentada e o histórico reflete a nova ordem.
Requisitos	RF24, RF18

4.3.4. CU04 — Configurar Unidades e Preferências

Tabela 7 - Caso de uso CU04 — Configurar Unidades e Preferências

Identificador	CU04
Nome	Configurar unidades de medida, tema e idioma
Ator	Utilizador
Pré-condições	Nenhuma. As preferências têm valores por omissão (°C, km/h, tema claro, inglês).
Fluxo principal	1. O utilizador acede aos controlos de preferências no cabeçalho da interface. 2. O utilizador alterna a temperatura entre °C e °F. 3. O utilizador alterna a velocidade do vento entre km/h e mph. 4. O utilizador alterna o tema entre claro e escuro. 5. O utilizador alterna o idioma entre inglês e português. 6. O sistema atualiza imediatamente todos os valores apresentados e persiste a preferência em localStorage.
Fluxos alternativos	Cada passo 2-5 é independente e pode ser executado em qualquer ordem.
Pós condições	As preferências são aplicadas a todas as secções da interface e persistem entre sessões.
Requisitos	RF25, RF26, RF27, RNF09

4.3.5. Consultar a API via Swagger UI

Tabela 8 - Caso de uso CU05 - Consultar a API via

Identificador	CU05
Nome	Consultar e testar a API REST via documentação interativa
Ator	Programador / integrador
Pré-condições	O <i>backend</i> está em funcionamento e acessível em /docs.
Fluxo principal	1. O programador acede a http://localhost:8000/docs. 2. O Swagger UI apresenta todos os <i>endpoints</i> agrupados por fornecedor. 3. O programador expande um <i>endpoint</i>, preenche os parâmetros e clica em “Execute”. 4. O sistema executa o pedido e apresenta a resposta JSON completa, incluindo cabeçalhos e código de estado HTTP.
Fluxos alternativos	3a. O programador acede a /redoc para uma vista de documentação alternativa, sem capacidade de execução.
Pós condições	O programador obtém a resposta do <i>endpoint</i> selecionado e compreende o formato dos dados devolvidos.
Requisitos	RF16, RNF03

4.4. Restrições e Limitações Identificadas

A análise da documentação e dos contratos das APIs dos fornecedores revelou um conjunto de restrições que condicionaram as decisões de arquitetura e implementação. Estas restrições são apresentadas por fornecedor.

4.4.1. Open-Meteo

O Open-Meteo não impõe restrições significativas à integração. A API é gratuita, sem autenticação, e suporta até 10.000 pedidos diários em utilização não comercial. A resolução de cidades depende do serviço de geocodificação associado, que não suporta geocodificação inversa (coordenadas → nome de cidade), sendo necessário recorrer ao Nominatim da OpenStreetMap para essa operação. Adicionalmente, a cobertura geográfica com seleção automática de fornecedor está limitada a 15 países no âmbito do presente projeto.

4.4.2. IPMA

O IPMA disponibiliza apenas previsões diárias com um horizonte máximo de 5 dias. Não existe *endpoint* de condições atuais nem de previsão horária. O catálogo de cidades está limitado às localidades para as quais o IPMA produz previsões específicas, o que exclui algumas localidades de menor dimensão. A resposta da API segue um formato JSON específico com campos numéricos para o tipo de tempo (*idWeatherType*) que necessitam de mapeamento para descrições textuais via tabela local.

4.4.3. AEMET

A AEMET impõe as restrições mais significativas de entre todos os fornecedores integrados:

- **Ausência de condições atuais:** a plataforma *OpenData* da AEMET não disponibiliza observações em tempo real, apenas previsões.
- **Fluxo de dois passos:** todos os *endpoints* de previsão devolvem, num primeiro pedido, um objeto de meta-dados contendo um URL temporário (*datos*) que deve ser consultado num segundo pedido HTTP para obter os dados efetivos.
- **Codificação ISO-8859-1:** as respostas da AEMET são codificadas em ISO-8859-1 (latin-1) e não em UTF-8, o que requer decodificação explícita de *r.content* antes do *parsing* JSON. Este comportamento *manifestouse* como um erro de *runtime* no processamento de localidades com caracteres especiais do espanhol (ex.: o *byte* 0xBA correspondente ao símbolo de ordinal masculino “o”).
- **Identificação por código INE:** as localidades espanholas são identificadas pelo código do *Instituto Nacional de Estadística* (INE), e não pelo nome. A resolução de um nome de cidade para o código INE correspondente requer a manutenção de um catálogo de municípios obtido da própria API.

4.4.4. CWA

A CWA impõe as seguintes restrições relevantes:

- **Datasets por condado:** os dados de previsão não estão disponíveis num *dataset* único para todo o país. Cada um dos 22 condados e cidades de Taiwan possui o seu próprio identificador de *dataset* (por exemplo, F-D0047-061 para Taipei). O identificador global F-D0047-

093, que sugeria cobertura de todos os condados, revelou-se inválido durante os testes, devolvendo HTTP 404 na generalidade dos pedidos, pelo que foi necessário construir e manter um registo local com o identificador correto para cada condado.

- **Velocidade do vento textual:** as previsões da CWA não incluem a velocidade do vento em unidades numéricas (km/h), mas sim em classes textuais em mandarim, impossibilitando a conversão para milhas por hora.
- **Valores sentinela:** os dados de observação de estações utilizam valores sentinela (-99, -9999, 9999) para indicar dados em falta ou inválidos, que devem ser filtrados e convertidos para null antes de serem devolvidos ao cliente.

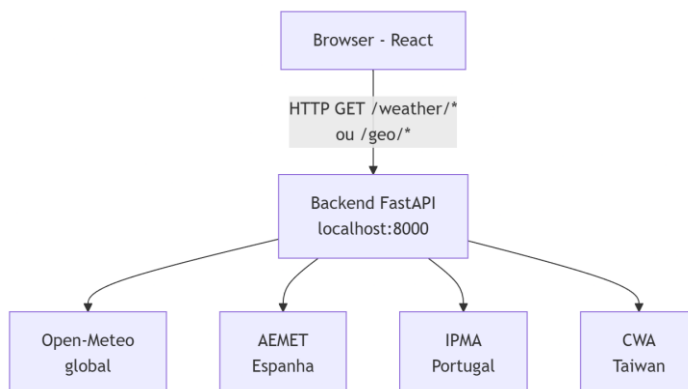
5. Arquitetura e Implementação

O presente capítulo descreve as decisões de arquitetura do sistema **Local Weather** e a sua implementação técnica. O sistema é composto por dois subsistemas independentes: um *backend* REST em Python e um *frontend* React que comunicam exclusivamente através de HTTP

5.1. Visão e Arquitetura Geral

O sistema segue uma arquitetura cliente-servidor em três camadas lógicas: o *frontend* React que corre no *browser* do utilizador, o *backend* FastAPI que expõe a API REST, e as APIs externas dos fornecedores meteorológicos. O fluxo de dados segue sempre a mesma direção: o *browser* comunica apenas com o *backend* próprio, que por sua vez comunica com os fornecedores externos. Os fornecedores externos nunca são acedidos diretamente pelo *browser*, evitando assim problemas de CORS e ocultando as chaves de API.

Figura 1 - Fluxo de comunicação entre os componentes do sistema



A decisão de qual fornecedor utilizar para cada pesquisa é tomada no *frontend*: com base no código de país da cidade selecionada pelo utilizador, o *hook useWeather* determina qual o *endpoint* do *backend* a invocar e quais as chamadas a realizar em paralelo.

5.2. *Backend* - Estrutura dos Módulos

O *backend* está organizado em quatro camadas horizontais, cada uma com uma responsabilidade bem definida, como mostrado no Apêndice B. A separação entre camadas garante que a lógica de comunicação HTTP com os fornecedores externos está totalmente isolada da lógica de roteamento da API, facilitando tanto a testabilidade como a extensão futura.

Camada de modelos (*app/models/*): define os esquemas Pydantic que tipificam as respostas de cada *endpoint*. Cada fornecedor tem o seu próprio sub-diretório com esquemas independentes, refletindo exatamente a estrutura de dados que cada API oferece. Não existe tentativa de unificar os modelos entre fornecedores, o IPMA tem cinco dias e temperatura mínima/máxima como *strings*; o OpenMeteo tem até 16 dias e valores numéricos com unidades explícitas, e esta diferença é preservada.

Camada de adaptadores (*app/providers/*): encapsula toda a comunicação HTTP com as APIs externas. Cada adaptador é completamente isolado dos restantes e expõe apenas funções assíncronas de alto nível (ex.: `fetch_daily(municipio_id)`, `fetch_forecast(global_id_local)`). Os *routers* não fazem chamadas HTTP diretas apenas invocam estas funções.

Camada de routers (*app/routers/*): define os *endpoints* FastAPI, valida os parâmetros de entrada via Pydantic, invoca os adaptadores correspondentes, e serializa as respostas para JSON. Cada fornecedor tem o seu próprio módulo de *router*, registado com um prefixo distinto em *main.py*.

Módulos transversais: *config.py* carrega as chaves de API a partir do ficheiro *.env* via *pydantic-settings*; *geocoding.py* implementa a resolução de nomes de cidades em coordenadas via a API de geocodificação do Open-Meteo, partilhada pelos routers que precisam de coordenadas geográficas.

5.3. Adaptadores por Fornecedor

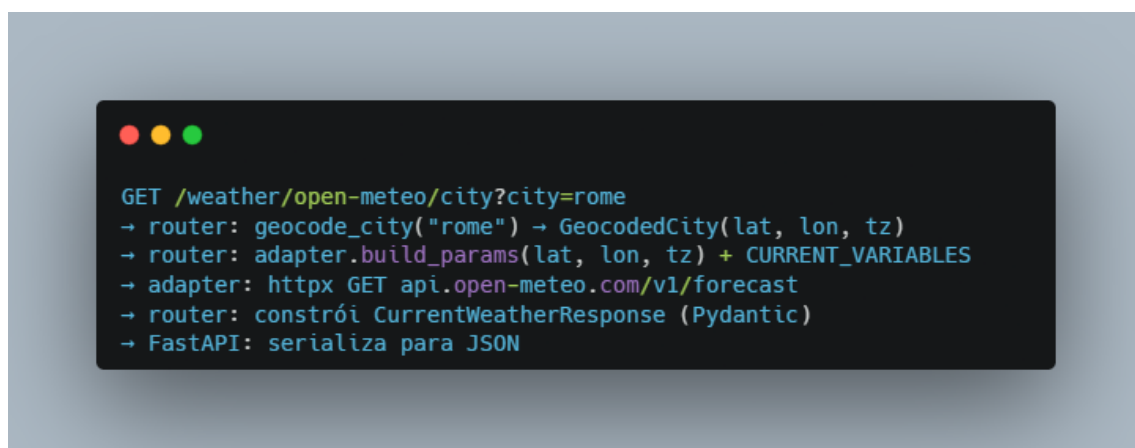
5.3.1. Open-Meteo

O adaptador Open-Meteo é o mais simples dos quatro. A API do Open-Meteo devolve dados diretamente num único pedido GET, sem autenticação, sem fluxo multi-fase e sem anomalias de codificação.

O módulo `providers/open_meteo/adapter.py` define três conjuntos de variáveis: `CURRENT_VARIABLES`, `HOURLY_VARIABLES` e `DAILY_VARIABLES`; que são enviados como parâmetro `current/hourly/daily` nos pedidos à API. A função `build_params()` constrói os parâmetros base partilhados pelas três variantes (latitude, longitude, fuso horário, unidades), e `fetch()` realiza o pedido HTTP com tratamento de erros de rede, *timeout* (15 s) e erros HTTP.

O fluxo completo para o *endpoint* de condições atuais é ilustrativo da cadeia de responsabilidades na Figura 2.

Figura 2 - Fluxo do endpoint "Condições atuais"



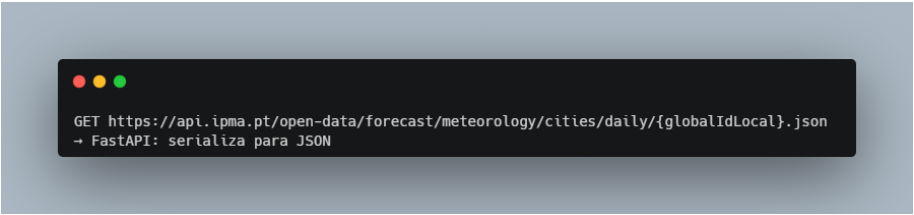
5.3.2. IPMA

A integração do IPMA tem uma particularidade de arquitetura: as localizações não são identificadas por nome ou por coordenadas, mas por um identificador inteiro fixo (`globalIdLocal`). O módulo `providers/ipma/cities.py` mantém um registo local de todas as localidades suportadas pelo IPMA (capitais de distrito, ilhas dos Açores e da Madeira), com os seus identificadores, coordenadas e designação do distrito.

A pesquisa de cidades é implementada no *router* (`routers/ipma/forecast.py`) como uma filtragem em memória sobre este catálogo estático: dado o nome pesquisado pelo utilizador, o sistema percorre o catálogo e devolve as localidades com correspondência parcial, insensível a maiúsculas. Esta abordagem evita uma chamada de rede extra para cada pesquisa e garante resultados instantâneos.

O adaptador (`providers/ipma/adapter.py`) realiza um único pedido **GET** ao *endpoint* de previsão do IPMA:

Figura 3 - IPMA *adapter*



```
GET https://api.ipma.pt/open-data/forecast/meteorology/cities/daily/{globalIdLocal}.json  
→ FastAPI: serializa para JSON
```

5.3.3. AEMET

A integração da AEMET é a mais complexa do sistema, pelas três razões já identificadas na análise de requisitos: o fluxo de dois passos, a codificação ISO-8859-1 e a identificação por código INE.

Fluxo de dois passos. Todos os *endpoints* de previsão da AEMET seguem um protocolo de dois pedidos HTTP. O primeiro pedido devolve um objeto de metadados com o campo "*datos*" contendo um URL temporário gerado pelo servidor. O segundo pedido é feito a esse URL para obter os dados efetivos de previsão. A função `_fetch_two_phase()` encapsula este protocolo de forma transparente:

Figura 4 - AEMET fluxo de duas etapas



```
async def _fetch_two_phase(metadata_url: str) -> list:  
    meta = await _get(metadata_url) # fase 1  
    datos_url = meta.get("datos")  
    # fase 2 - sem api_key, o URL já é autenticado  
    r = await client.get(datos_url)  
    return _decode(r)
```

Descodificação ISO-8859-1. A função `_decode()` lê os *bytes* brutos da resposta e descodifica-os explicitamente em latin-1 antes de fazer *parsing* do JSON, contornando o comportamento por omissão do `httplib` que assume UTF8:

Figura 5 - Decodificação ISO-8859-1

Catálogo de municípios. O módulo `providers/aemet/municipios.py` implementa um cache em memória do catálogo de municípios espanhóis (aproximadamente 8.000 entradas), populado na primeira invocação via chamada à API da AEMET. O *cache* é protegido por um `asyncio.Lock()` para evitar chamadas concorrentes redundantes durante a inicialização. A pesquisa de municípios por nome é uma filtragem em memória sobre este *cache*.

5.3.4. CWA Taiwan

O adaptador CWA utiliza três *datasets* distintos da plataforma de dados abertos da CWA, cada um com o seu identificador:

- **O-A0001-001** - observações atuais das estações meteorológicas automáticas;
- **F-C0032-001** - previsão de 36 horas por condado (3 períodos de 12 horas);
- **F-D0047-0XX** - previsão de 7 dias por condado (identificador específico para cada um dos 22 condados).

A necessidade de identificadores por condado para as previsões a 7 dias resultou de uma descoberta durante a integração: o identificador F-D0047-093, que a documentação sugeria como identificador global para todo o território de Taiwan, devolvia **HTTP 404** na generalidade dos pedidos. A solução adotada foi construir e manter o registo `CWA_LOCATIONS` em `providers/cwa/locations.py`, com o identificador de *dataset* correto para cada um dos 22 condados e cidades de Taiwan.

Cada entrada do registo inclui o nome em chinês, o nome em inglês, uma lista de *aliases* para pesquisa (incluindo versões alternativas da romanização e nomes históricos como “Pescadores” para Penghu ou “Quemoy” para Kinmen), e os identificadores de *dataset* para previsão de 2 dias.

5.3.5. Geocodificação e Geolocalização

O módulo `app/geocoding.py` implementa a resolução de nomes de cidades em coordenadas geográficas. A função assíncrona `geocode_city(city, country_hint)` consulta a API de geocodificação do Open-Meteo, que aceita um nome livre e devolve uma lista de resultados ordenada por relevância, cada um com `country_code`, `latitude`, `longitude` e `timezone`.

A lógica de seleção do resultado correto percorre a lista e devolve o primeiro resultado cujo `country_code` está no conjunto de países suportados e, opcionalmente, corresponde à dica de país fornecida. A dica de país é aplicada como filtro do lado do cliente nunca é anexada ao termo de pesquisa enviado à API porque testes empíricos demonstraram que a API de geocodificação do Open-Meteo devolve resultados vazios quando o nome do país é incluído no campo `name`. A classe `GeocodedCity` encapsula o resultado e normaliza o `country_code` para maiúsculas.

A funcionalidade de geolocalização automática no *frontend* utiliza a API do *browser* `navigator.geolocation` para obter as coordenadas GPS do utilizador. As coordenadas são enviadas ao *endpoint* `GET /geo/reverse`, que as resolve num nome de cidade e país usando o serviço Nominatim da OpenStreetMap. Este intermediário no *backend* é necessário porque o Nominatim não envia os cabeçalhos CORS necessários para ser chamado diretamente pelo browser. O fuso horário é resolvido em paralelo via um pedido mínimo à API de previsão do Open-Meteo com o parâmetro `timezone=auto`.

5.4. Frontend - Arquitetura React

O *frontend* está organizado em cinco categorias de módulos: componentes de interface, *hooks* de lógica, contextos de estado global, utilitários e internacionalização; como mostrado no Apêndice C.

5.4.1. Pesquisa Unificada de Cidades

O *hook* `useAllLocations(query)` no ficheiro `utils/useLocations.js` implementa a pesquisa em tempo real sobre todos os fornecedores simultaneamente. Quando o utilizador digita na barra de pesquisa, as quatro fontes são consultadas em paralelo com `Promise.all()`:

- **Open-Meteo:** filtragem síncrona sobre uma lista estática de 50 cidades curadas em `utils/locations.js`;

- **IPMA:** filtragem em memória sobre o catálogo de cidades portuguesas, carregado do *backend* na primeira pesquisa e cacheado em variável de módulo;
- **CWA:** filtragem em memória sobre os 22 condados de Taiwan, carregados do *backend* na primeira pesquisa;
- **AEMET:** pesquisa ao vivo via GET `/weather/aemet/cities?city={query}`, com *debounce* de 300 ms e mínimo de 2 caracteres antes de iniciar a chamada (dado o volume de 8.000 municípios).

Os resultados são fundidos numa lista única, deduplicados por chave `provider:label`, e limitados a 30 entradas. Cada resultado inclui bandeira do país, nome da cidade, país, e cor do fornecedor exibida como um pequeno distintivo junto ao resultado.

5.4.2. Seleção do Fornecedor e Obtenção de Dados

O *hook* `useWeather` no ficheiro `hooks/useWeather.js` é responsável por toda a lógica de obtenção de dados meteorológicos. Quando o utilizador seleciona uma cidade, o *hook* recebe o fornecedor já identificado (`provider`) e executa as chamadas adequadas:

- **AEMET:** duas chamadas paralelas (previsão diária e horária) via `Promise.allSettled()`, permitindo que uma falhe sem cancelar a outra;
- **IPMA:** uma única chamada à previsão diária;
- **CWA:** três chamadas paralelas (condições atuais e 36h) via `Promise.allSettled()`;
- **Open-Meteo:** três chamadas paralelas (condições atuais, horária e diária) via `Promise.all()`.

O uso de `Promise.allSettled()` para AEMET e CWA é intencional: permite que o sistema apresente dados parciais (por exemplo, previsão diária) mesmo que uma das chamadas falhe, em vez de mostrar um erro geral. O `Promise.all()` para o Open-Meteo é adequado porque as três chamadas têm igual probabilidade de falha e o Open-Meteo é o fornecedor mais estável.

5.4.3. Estado Global - Contextos React

O frontend utiliza dois contextos React para estado global:

UnitsContext gere as preferências de unidades de medida (temperatura em °C ou °F; velocidade do vento em km/h ou mph). As preferências são guardadas em `localStorage` sob a chave `localweather.units` e carregadas na inicialização. O contexto expõe funções de conversão tipificadas (`fmtTemp`, `fmtWind`, `cvtTemp`) que são consumidas diretamente pelos componentes de visualização, garantindo que uma alteração de unidades se reflete imediatamente em todos os valores apresentados sem necessidade de novo pedido ao *backend*.

LanguageContext gere o idioma da interface (inglês ou português). As *strings* de interface estão centralizadas em `i18n/translations.js` e acedidas via a função `t(key)`, evitando *strings* dispersas nos componentes.

5.4.4. Visão Gráfica

A visualização gráfica das previsões é realizada pelos componentes `Hourly\allowbreakTempChart` e `DailyTempChart`, implementados em `components/TemperatureChart.jsx` com a biblioteca MUI X Charts. O gráfico horário utiliza um `LineChart` para a curva de temperatura, com um eixo Y secundário à direita para a probabilidade de precipitação quando disponível. O gráfico diário utiliza um `BarChart` com barras agrupadas para temperatura máxima (laranja) e mínima (azul).

Uma restrição relevante da biblioteca foi identificada durante a implementação: os componentes MUI X Charts processam os valores de cor internamente através do módulo `d3-color`, que não consegue interpretar variáveis CSS do tipo `var(-accent)`. A tentativa de chamar `.brighter()` sobre o valor `null` resultante gerava um erro de *runtime*. A solução adotada foi utilizar valores hexadecimais explícitos para todas as cores passadas aos componentes MUI.

5.5. Comunicação Frontend-Backend

Toda a comunicação do frontend com o *backend* é centralizada no ficheiro `utils/api.js`, que exporta um objeto API com métodos organizados por fornecedor:

Figura 6 - JSON comunicação *frontend* com *backend*

```
export const api = {
  openMeteo: {
    current: (city, country) => get(`/weather/open-meteo/city?...`),
    hourly: (city, country, days) => get(`/.../hourly?...`),
    daily: (city, country, days) => get(`/.../daily?...`),
  },
  aemet: {
    cities: (city) => get(`/weather/aemet/cities?city=...`),
    daily: (municipio_id) => get(`/weather/aemet/forecast/daily?...`),
    hourly: (municipio_id) => get(`/weather/aemet/forecast/hourly?...`),
  },
  ipma: { cities: () => ..., forecast: (city) => ... },
  cwa: { current: ..., forecast36h: ..., daily: ... },
};
: { cities: () => ..., forecast: (city) => ... },
cwa: { current: ..., forecast36h: ..., daily: ... },
}
```

A função base `get(path)` usa o *fetch* nativo do *browser*, verificando o código de estado HTTP e lançando um erro com o campo `detail` da resposta JSON em caso de falha. Em ambiente de desenvolvimento, o `package.json` configura um *proxy* para `http://localhost:8000`, evitando problemas de **CORS** durante o desenvolvimento local sem necessidade de configurar cabeçalhos no *backend* para este caso.

5.6. Principais Desafios de Implementação

5.6.1. Codificação ISO-8859-1 das Respostas AEMET

O primeiro desafio significativo durante a integração da AEMET foi um erro de *parsing* JSON que se manifestava apenas para localidades com caracteres especiais espanhóis. A causa raiz foi identificada como uma incompatibilidade de codificação: a AEMET codifica as respostas em ISO-8859-1 enquanto o `httpx` assume UTF-8. O *byte* 0xBA (ordinal masculino “o”), presente em nomes de lugares como “1º de Mayo”, era interpretado incorretamente, corrompendo o JSON antes do *parsing*. A solução foi ler os *bytes* brutos com `r.content` e decodificá-los explicitamente com `.decode("latin-1")` antes de chamar `json.loads()`, em vez de usar `r.json()` diretamente.

5.6.2. Identificadores de *Dataset* da CWA

A documentação da CWA referia o identificador F-D0047-093 como o *dataset* de previsão de 7 dias para todo o território de Taiwan. Os testes de integração com a

API real revelaram que este identificador devolvia HTTP 404 de forma consistente. A investigação identificou que a CWA organiza os dados de previsão de 7 dias por condado, com um identificador específico para cada um dos 22 condados e cidades de Taiwan (de F-D0047-001 a F-D0047-087, números ímpares). A solução foi construir e manter o registo CWA_LOCATIONS com o identificador correto para cada localização.

5.6.3. CORS na Geocodificação Inversa

A funcionalidade de geolocalização automática requer a conversão de coordenadas GPS num nome de cidade. O Nominatim da OpenStreetMap é o serviço mais adequado para esta operação (geocodificação inversa real, sem necessidade de chave), mas não envia o cabeçalho *Access-Control-AllowOrigin* nas respostas, o que impede as chamadas diretas a partir de um *browser*. A solução foi criar o *endpoint* GET */geo/reverse* no *backend*, que funciona como *proxy*: recebe as coordenadas do *frontend*, chama o Nominatim *serverside* (onde não há restrições CORS), e devolve o resultado ao *browser*.

5.6.4. Mocking Assíncrono nos Testes

Durante a escrita dos testes de integração, verificou-se que a substituição do `httpx.AsyncClient` por `AsyncMock` produzia erros do tipo `AttributeError: 'coroutine' object has no attribute 'get'`. A causa foi identificada: os métodos `r.json()` e `r.raise_for_status()` são chamados de forma síncrona nos adaptadores, mesmo que o cliente seja assíncrono. Usar `AsyncMock` para estes métodos envolvia-os em *coroutines* que nunca eram aguardadas. A solução foi adotar `MagicMock` para estes dois métodos específicos, mantendo `AsyncMock` apenas para o método `client.get()` que é efetivamente aguardado com `await`.

6. Testes e Validação

O presente capítulo descreve a estratégia de testes adotada para o sistema **Local Weather**, os tipos de teste implementados, os casos de teste mais relevantes e os resultados de cobertura obtidos. A suite de testes cobre exclusivamente o *backend* FastAPI por ser o componente com lógica de negócio verificável de forma isolada e onde os erros de *parsing* e comunicação com as APIs externas têm maior impacto.

6.1. Estratégia de Testes

A estratégia adotada organiza-se em duas camadas complementares: testes unitários e testes de integração; e segue três princípios fundamentais que guiaram

tanto a estrutura dos testes como a refatorização do código de produção para os suportar.

6.1.1. Ausência de Chamadas HTTP Reais

Nenhum teste realiza chamadas HTTP efetivas às APIs dos fornecedores externos. Todas as respostas são simuladas através de objetos *mock* construídos com *payloads* realistas que reproduzem a forma exata das respostas reais de cada API. Esta decisão garante que os testes são determinísticos, independentes de chaves de API e de disponibilidade de serviços externos, e executáveis em qualquer ambiente sem configuração adicional.

6.1.2. Padrão Correto de *Mock* do *httpx*

Os adaptadores do sistema utilizam o cliente HTTP assíncrono `httpx.AsyncClient`. A simulação deste cliente revelou uma subtilidade importante: os métodos `r.json()` e `r.raise_for_status()` são chamados de forma síncrona nos adaptadores, apesar de o cliente ser assíncrono. Utilizar `AsyncMock` para estes métodos criaria objetos *coroutine* em vez de valores, resultando em erros do tipo `AttributeError: 'coroutine' object has no attribute 'get'`. O padrão correto, estabelecido na *fixture* partilhada `make_httpx_response()`, utiliza `MagicMock` para estes métodos:

Figura 7 - Proteção de *mock* do *httpx*



```
def make_httpx_response(payload, encoding="utf-8"):

    r = MagicMock()
    r.json = MagicMock(return_value=payload) # síncrono
    r.raise_for_status = MagicMock() # síncrono
    r.content = json.dumps(payload).encode(encoding)

    return r
```

6.1.3. Regra do *Patch* no Local de Uso

O Python resolve referências de símbolos no momento da importação. Ao fazer *patch* de uma função, é necessário substituí-la no módulo onde é utilizada, não no módulo onde é definida. Por exemplo, para testar o *router* do Open-Meteo, a função de geocodificação é substituída em `app.routers`, `open_meteo.city.geocode_city` e não em `app.geocoding.geocode_city`, pois é no *router* que a referência local foi criada no momento da importação. O mesmo princípio aplica-se aos adaptadores da CWA nos testes de integração, que são substituídos em `app.routers.cwa.forecast.*` em vez de `app.providers.cwa.adapter.*`.

6.2. Testes Unitários

Os testes unitários verificam módulos individuais de forma isolada, substituindo todas as dependências externas por *mocks*. Estão organizados em quatro ficheiros, um por domínio funcional.

6.2.1. Geocodificação (`test_geocoding.py`)

Este módulo testa a classe `GeocodedCity` e a função assíncrona `geocode_city()`, que são responsáveis por converter nomes de cidades em coordenadas geográficas.

Os testes da classe `GeocodedCity` verificam o mapeamento correto de todos os campos a partir do JSON da API, a normalização automática do `country_code` para maiúsculas, a resolução do nome do país a partir do código ISO (independentemente do valor devolvido pela API), o valor por omissão de "UTC" para o fuso horário quando ausente, e o método `is_supported()` para todos os códigos de país suportados e não suportados.

Os testes de `geocode_city()` cobrem os seguintes cenários:

- Resolução bem-sucedida com retorno correto de nome, coordenadas, fuso horário e região administrativa;
- Dica de país (*country hint*) com correspondência exata, parcial e insensível a maiúsculas;
- Verificação de que a dica de país nunca é anexada ao termo de pesquisa enviado à API (regra de geocodificação *client-side*);
- Resultados vazios - HTTP 404;
- Todos os resultados em países não suportados - HTTP 400;

- Dica de país desconhecida - HTTP 400;
- Cidade encontrada noutro país que não o da dica - HTTP 404;
- *Timeout* da API de geocodificação - HTTP 504;
- Erros HTTP (500) e de rede (*RequestError*) - HTTP 502.

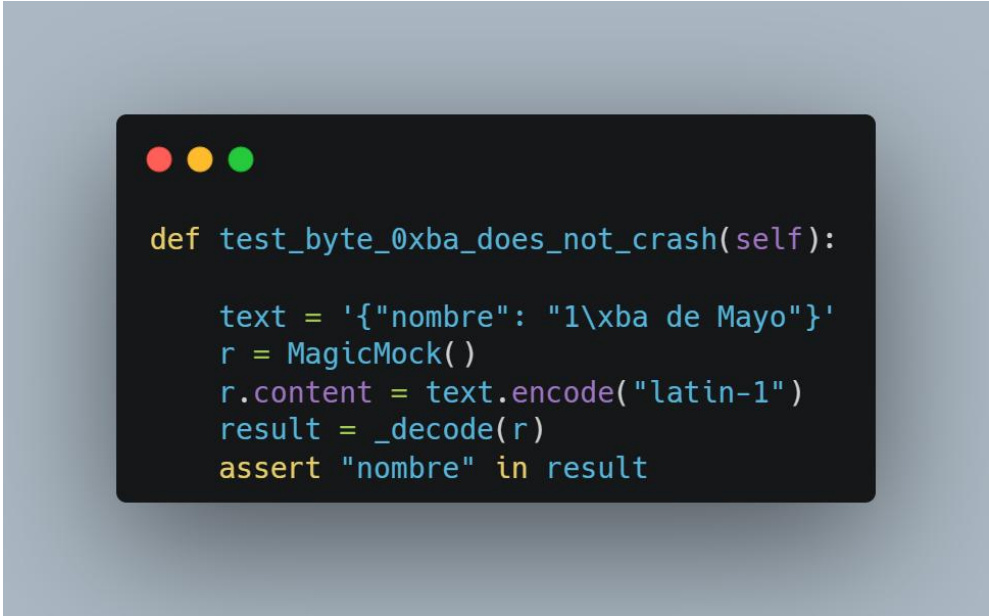
6.2.2. Fornecedor AEMET (test_aemet.py)

Este módulo é o mais extenso dos testes unitários (44 testes) e reflete a maior complexidade do adaptador AEMET. Está dividido em seis classes de teste:

TestAEMETMunicipio - verifica a limpeza do prefixo "id" do código INE: "id28079" deve ser normalizado para "28079".

TestDecode - testa a função `_decode()`, que é o ponto central de tratamento da codificação ISO-8859-1 das respostas AEMET. Os casos incluem dicionários simples, *payloads* do tipo lista, caracteres especiais do espanhol ("ñ", "á", "é") e, em particular, o byte 0xBA (ordinal masculino "o") que provocou a falha original na integração e que constitui um teste de regressão:

Figura 8 - Teste da função `_decode()`

A screenshot of a code editor with a dark background and light-colored text. The code is a Python test function named `test_byte_0xba_does_not_crash`. It defines a `text` variable with a JSON string containing a byte escape sequence, creates a `MagicMock` object, encodes the text, calls the `_decode` function, and asserts that the key "nombre" is in the result.

```
def test_byte_0xba_does_not_crash(self):  
  
    text = '{"nombre": "l\xba de Mayo"}'  
    r = MagicMock()  
    r.content = text.encode("latin-1")  
    result = _decode(r)  
    assert "nombre" in result
```

TestFetchTwoPhase - testa o fluxo de dois passos da API AEMET: o primeiro pedido deve devolver um objeto com o campo "*datos*" contendo um URL; o segundo pedido deve usar esse URL para obter os dados efetivos. Os casos cobrem o fluxo de sucesso, a verificação de que o segundo pedido usa o URL correto, o código de estado 404 na resposta de meta-dados, a ausência do campo "*datos*", *timeout* na fase 1, e erros HTTP 401.

TestStrOrNone, TestPvHelper, TestWindHelper, TestSkyHelper - testam os auxiliares de *parsing* que convertem os campos do JSON AEMET para os modelos internos, incluindo casos limite como *strings* vazias (devem produzir *null*), zero como valor numérico válido, e listas vazias.

TestParseDailyDay e TestParseHourlyEntries - testam o *parsing* completo de um dia de previsão diária e das entradas horárias, incluindo o mapeamento das janelas de rajada máxima (*rachaMax*) às horas individuais e o tratamento de listas de campos ausentes.

6.2.3. Fornecedor IPMA (test_ipma.py)

Os 21 testes deste módulo distribuem-se por três classes:

TestIPMACities - verifica a integridade do catálogo local de cidades portuguesas: todos os campos obrigatórios presentes, ausência de identificadores duplicados, contagem mínima de cidades, índice correto por *globalIdLocal*, cobertura dos arquipélagos dos Açores e da Madeira, e pesquisa por nome exato, insensível a maiúsculas e parcial.

TestIPMAAdapter - testa o adaptador HTTP com *mocks*, cobrindo o caminho de sucesso, a verificação do URL correto com o identificador da cidade, a extensão *.json* obrigatória no URL, e o mapeamento de erros HTTP (404 → 404, 500 → 502, *timeout* → 504, *RequestError* → 502).

TestParseEntry - testa a função *_parse_entry()* com os três tipos de tempo do *fixture* (parcialmente nublado, aguaceiros, céu limpo), campos nulos, código de tipo de tempo desconhecido (deve devolver *None* na descrição), e o código 0 (“Sem informação”).

6.2.4. Fornecedor CWA (test_cwa.py)

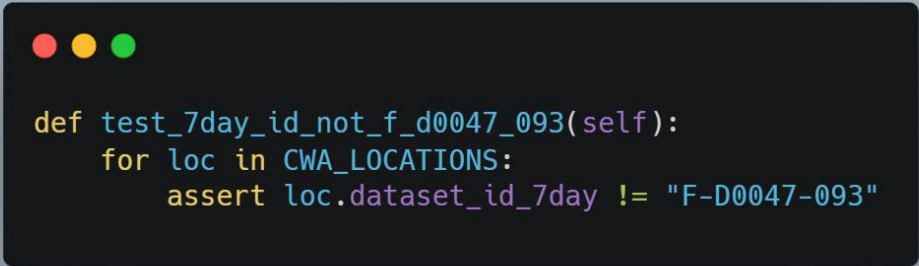
Com 55 testes, este módulo reflete a complexidade da integração CWA. Está dividido em seis classes:

TestFindLocation - testa a resolução de localizações por nome inglês exato e insensível a maiúsculas, por nome chinês, por *aliases* (“taipei”, “kaohsiung”, “matsu”, “pescadores”, “quemoy”), por correspondência parcial, e a distinção entre cidades e condados homónimos (ex.: “Hsinchu City” vs. “Hsinchu County”). Inclui também um teste que verifica que todos os 22 condados e cidades de Taiwan são localizáveis tanto pelo nome inglês como pelo nome chinês.

TestSearchLocations - testa a pesquisa parcial com múltiplos resultados, pesquisa em chinês, insensibilidade a maiúsculas, e a inclusão de *aliases* nos resultados.

TestDatasetIds - testa a integridade dos identificadores de *dataset* CWA: presença dos 22 identificadores, valores corretos para Taipei e Kaohsiung, ausência de duplicados, diferença entre identificadores de 3 dias e 7 dias para a mesma localização, formato correto com prefixo F-D0047-, e em particular a verificação de que nenhuma localização usa o identificador F-D0047-093, que se revelou inválido durante os testes de integração com a API real:

Figura 9 - Teste de *datasetsIds*



```
def test_7day_id_not_f_d0047_093(self):  
    for loc in CWA_LOCATIONS:  
        assert loc.dataset_id_7day != "F-D0047-093"
```

TestSafeFloat - testa a função de conversão segura de valores numéricos, com especial atenção aos valores sentinela (-99, -9999, 9999) que a CWA utiliza para indicar dados em falta e que devem ser convertidos para *null*, bem como ao tratamento de zero como valor válido, *strings* inválidas e entradas nulas.

TestElementExtractors e **TestResolveLocation** - testam os auxiliares de extração de elementos do JSON CWA e a resolução de localizações com tratamento de erros HTTP 404.

6.3. Testes de Integração

Os testes de integração exercitam o sistema completo através do ciclo de pedido-resposta do FastAPI, utilizando o *TestClient* da biblioteca Starlette. Cada pedido de teste percorre a cadeia completa: validação de parâmetros Pydantic, lógica do router, chamada ao adaptador (com *mocks* das respostas HTTP externas), *parsing* e serialização da resposta. O ficheiro `test_endpoints.py` contém 51 testes distribuídos por cinco classes.

6.3.1. Testes de Saúde e Infra-estrutura (*TestHealth*)

Verificam que o *endpoint* `/health` devolve HTTP 200 com a lista correta de fornecedores, que GET `/` redireciona para `/docs`, e que o esquema OpenAPI está acessível em `/openapi.json`.

6.3.2. Testes Open-Meteo (*TestOpenMeteoEndpoints*)

Cobrem os três *endpoints* Open-Meteo (`/city`, `/city/hourly`, `/city/daily`) com os seguintes cenários: resposta HTTP 200 com *payload* bem formado, presença do bloco `resolved_location` na resposta, verificação dos campos de condições atuais, funcionamento da dica de país, forma correta dos dados horários e diários, e respeito do parâmetro `forecast_days`.

A *fixture* de geocodificação para estes testes substitui `geocode_city` directamente no módulo do router com uma função assíncrona que devolve um objeto `GeocodedCity` pré-construído, evitando qualquer chamada HTTP real:

Figura 10 - Teste de endpoints Open-Meteo



```
@pytest.fixture(autouse=True)
def mock_geocoding(self):

    async def fake_geocode(city, country_hint=None):
        return GeocodedCity(GEOCODING_ROME["results"][0])

    with patch("app.routers.open_meteo.city.geocode_city",
               side_effect=fake_geocode):
        yield
```

6.3.3. Testes IPMA (TestIPMAEndpoints)

Cobrem os dois *endpoints* IPMA: listagem de cidades com verificação do formato da resposta, pesquisa por nome, e previsão diária com verificação dos campos de data, temperatura e tipo de tempo.

6.3.4. Testes AEMET (TestAEMETEndpoints)

Cobrem os três *endpoints* AEMET. A *fixture* substitui as funções `fetch_daily` e `fetch_hourly` diretamente no módulo do *router* com `AsyncMock`, devolvendo os *payloads* realistas definidos no *conftest*. Os casos testam a pesquisa de municípios, a previsão diária (incluindo temperaturas máxima e mínima, estado do céu e UV), e a previsão horária (temperatura, humidade, vento e probabilidade de precipitação).

6.3.5. Testes CWA (TestCWAEndpoints)

Cobrem os quatro *endpoints* CWA. As funções de acesso à API (`fetch_current_observations`, `fetch_36h_forecast`, `fetch_7day_forecast`) são substituídas no módulo do *router* não no módulo do adaptador por `AsyncMock`. Os casos incluem:

- Observações atuais com verificação da estação e dos campos meteorológicos;
- Filtragem de valores sentinela nos dados de observação (temperatura - 99 deve ser devolvida como *null*);
- Previsão de 36 horas com verificação dos períodos e probabilidade de precipitação;
- Previsão diária com verificação dos nomes de bairro e dos períodos de previsão;
- Verificação de que o identificador de *dataset* correto é passado para `fetch_7day_forecast` (ex.: F-D0047-063 para Taipei).

6.4. Resultados e Cobertura

A suite de testes é executada com o comando:

```
pytest tests/ --cov=app --cov-report=term-missing
```

6.4.1. Sumário de Resultados

Tabela 9 - Sumário de testes

Ficheiro	Tipo	Testes	Classes
unit/test_geocoding.py	Unitário	25	2
unit/providers/test_aemet.py	Unitário	44	8
unit/providers/test_ipma.py	Unitário	21	3
unit/providers/test_cwa.py	Unitário	55	6
integration/test_endpoints.py	Integração	51	5
Total		196	24

Todos os testes passam com 0 falhas. A execução completa da suite demora aproximadamente 9 segundos, sem qualquer chamada HTTP a serviços externos.

6.4.2. Cobertura por Módulo

Tabela 10 - Cobertura de código por módulo do *backend*

Módulo	Linhas	Cobertas	Cobertura
app/config.py	10	10	100%
app/geocoding.py	56	56	100%
app/main.py	21	21	100%
app/models/open_meteo/schemas.py	111	111	100%
app/models/ipma/schemas.py	22	22	100%
app/models/aemet/schemas.py	66	66	100%
app/models/cwa/schemas.py	64	64	100%
app/providers/ipma/adapters.py	18	18	100%
app/providers/ipma/cities.py	15	15	100%
app/routers/open_meteo/city.py	30	30	100%
app/routers/ipma/forecast.py	32	32	100%
app/providers/cwa/locations.py	33	32	97%
app/routers/aemet/forecast.py	84	81	96%

app/routers/cwa/forecast.py	127	117	92%
app/providers/cwa/adapter.py	36	32	89%
app/providers/aemet/adapter.py	78	46	59%
app/providers/aemet/municipios.py	31	23	74%
app/routers/geo.py	40	14	35%
TOTAL	894	790	89%

6.4.3. Análise da Cobertura

A cobertura global de 89% é considerada satisfatória para um projeto deste âmbito. Os módulos com cobertura de 100% correspondem às camadas de modelos de dados, configuração e aos adaptadores e routers mais simples. Os módulos com cobertura mais baixa justificam-se pelos seguintes motivos:

- **app/providers/aemet/adapter.py (59%)** - as linhas não cobertas correspondem a tratamento de erros de rede em cenários de falha parcial que são de difícil simulação com o padrão de mock adotado, nomeadamente erros na fase 2 do fluxo de dois passos com erros HTTP específicos.
- **app/routers/geo.py (35%)** - o endpoint de geocodificação inversa (/geo/ reverse) realiza duas chamadas HTTP sequenciais (Nominatim e Open-Meteo para o fuso horário) e os seus ramos de tratamento de erros não foram totalmente cobertos pelos testes de integração atuais. Esta é a área com maior margem de melhoria na suite de testes.
- **app/providers/aemet/municipios.py (74%)** - as linhas não cobertas correspondem ao carregamento inicial do catálogo de municípios a partir da API real, que é substituído por mock nos testes.

A configuração de `asyncio_mode = "auto"` no `pyproject.toml` permite que todos os testes assíncronos sejam descobertos e executados automaticamente pelo `pytest-asyncio` sem necessidade de decoradores individuais, simplificando a escrita de novos testes.

7. Conclusões e Trabalho Futuro

O presente projeto alcançou os objetivos definidos na fase de análise, concretizando um sistema *full-stack* funcional para agregação de dados meteorológicos provenientes de múltiplos fornecedores nacionais. O *backend* expõe treze *endpoints* REST documentados via Swagger UI, cobre quatro fornecedores com modelos de dados heterogêneos, e é validado por uma suite de 196 testes automatizados com 89% de cobertura de código. O frontend apresenta os dados de forma clara e acessível, com pesquisa unificada, geolocalização automática, histórico de pesquisas, suporte a dois idiomas, unidades configuráveis e visualização gráfica.

Do ponto de vista técnico, o projeto demonstrou a viabilidade da arquitetura de *provider adapter* para a integração de APIs meteorológicas heterogêneas: a adição de cada novo fornecedor exigiu apenas a criação de um módulo adaptador isolado, sem modificação dos módulos existentes. Os desafios mais significativos encontrados a codificação ISO-8859-1 das respostas AEMET, o fluxo de dois passos da mesma API, a ausência de um identificador global funcional nos dados CWA, e as restrições CORS na geocodificação inversa foram todos resolvidos e documentados, constituindo conhecimento transferível para integrações futuras.

7.1. Limitações Atuais

O sistema apresenta as seguintes limitações no seu estado atual, reconhecidas como direções naturais para desenvolvimentos futuros:

- **Seleção do fornecedor no *frontend*:** a lógica de encaminhamento para IPMA, AEMET, CWA ou Open-Meteo está implementada no frontend, em vez de existir um *endpoint* unificado no *backend* que abstraia esta decisão. Isto cria uma dependência entre o frontend e os detalhes dos fornecedores que idealmente deveria estar encapsulada no *backend*.
- **Ausência de *caching*:** cada pedido do utilizador resulta em chamadas às APIs externas dos fornecedores, sem qualquer mecanismo de reutilização de respostas recentes. Dado que as previsões meteorológicas são atualizadas tipicamente de 1 a 6 horas, a ausência de cache implica latência desnecessária e uso excessivo das chaves de API.
- **Cobertura geográfica limitada:** o sistema cobre quatro fornecedores e quinze países Open-Meteo. Regiões como o Brasil, a Alemanha (com

API do DWD), a Finlândia ou o Reino Unido têm agências meteorológicas nacionais com APIs abertas que não estão integradas.

- **Ausência de autenticação:** o *backend* não implementa qualquer mecanismo de autenticação ou *rate limiting* próprio, expondo diretamente as chaves de API dos fornecedores a utilizações abusivas em ambiente de produção.
- **Validação exclusivamente local:** a aplicação foi testada apenas em ambiente de desenvolvimento local, sem validação em ambiente de produção nem medição de desempenho em condições reais de carga.

7.2. Trabalho Futuro

As propostas de trabalho futuro estão organizadas por ordem decrescente de impacto arquitetural, distinguindo entre extensões que melhorariam a arquitetura existente, extensões que ampliariam a cobertura geográfica, e funcionalidades novas orientadas ao utilizador.

7.2.1. *Endpoint* Unificado no *Backend*

A extensão com maior impacto arquitetural seria a implementação do *endpoint* GET `/weather/city?city={nome}`, que selecionaria automaticamente o melhor fornecedor com base no país da cidade pesquisada e devolveria uma resposta com esquema unificado, independentemente da fonte.

Esta extensão completaria o ciclo do padrão *Aggregator*: atualmente o sistema implementa os adaptadores individuais, mas delega a lógica de seleção ao frontend. Um *endpoint* unificado centralizaria esta responsabilidade no *backend*, tornando a API consumível por qualquer cliente (não apenas a interface React) sem necessidade de conhecer os fornecedores subjacentes. O frontend passaria a fazer um único pedido em vez de três a quatro pedidos paralelos, simplificando o *hook useWeather*.

O esquema de resposta unificado teria de acomodar as diferenças de disponibilidade entre fornecedores: os campos `current` e `hourly` seriam *null* para o IPMA (que não os disponibiliza), e o campo `current` seria *null* para a AEMET. Um bloco `provider_meta` na resposta descreveria as capacidades ativas, permitindo ao frontend adaptar a interface sem lógica condicional baseada no identificador do fornecedor.

7.2.2. Caching de Respostas com TTL

A introdução de uma camada de *caching* no *backend* reduziria a latência percebida pelo utilizador e protegeria as chaves de API de desgaste por uso excessivo. A estratégia mais adequada seria um cache em memória com TTL (*Time To Live*) diferenciado por tipo de dado:

- Condições atuais: TTL de 10 minutos (atualizadas frequentemente pelas estações de observação);
- Previsão horária: TTL de 1 hora;
- Previsão diária: TTL de 3 horas;
- Catálogo de municípios AEMET: TTL de 24 horas (muda raramente).

A implementação mais simples usaria a biblioteca *cachetools* com um *TTLCache* com chave composta por fornecedor e parâmetros do pedido. Para um cenário de maior escala, o Redis constituiria a alternativa adequada, permitindo partilha de cache entre múltiplas instâncias do *backend* e persistência entre reinicializações.

7.2.3. Expansão de Fornecedores

A arquitetura de adaptadores facilita a adição de novos fornecedores sem modificação do código existente. Os candidatos mais relevantes, por ordem de prioridade, são:

- **DWD (*Deutscher Wetterdienst*, Alemanha):** disponibiliza uma API de dados abertos com previsões de alta resolução baseadas no modelo ICON a 2 km, sem necessidade de chave de API e com documentação em inglês. Seria o primeiro fornecedor europeu de língua não descendente do latim a integrar o sistema, aumentando a cobertura para a Europa central.
- **FMI (*Finnish Meteorological Institute*, Finlândia):** disponibiliza uma API WFS (**Web Feature Service**) com dados de observação e previsão para a Escandinávia e regiões árticas, cobrindo um vazio geográfico significativo.
- **INMET (Instituto Nacional de Meteorologia, Brasil):** disponibiliza uma API REST pública com dados de estações de observação e boletins de previsão para o território brasileiro. A integração do INMET seria particularmente relevante dada a motivação pessoal que deu origem ao projeto, a necessidade de acompanhar condições meteorológicas em países lusófonos.

- **Met Office (Reino Unido):** disponibiliza a *Global Weather Open* API, com previsões de hora em hora para qualquer localização do globo a resoluções de até 2 km, tornando-a uma alternativa de alta resolução ao Open-Meteo para a cobertura global.

7.2.4. Observabilidade e *Dashboard* de Saúde dos Fornecedores

O *backend* inclui um *endpoint* GET `/health` que confirma o estado operacional da API, mas não mede a disponibilidade dos fornecedores externos. Uma extensão natural seria um *endpoint* GET `/health/providers` que realizasse um pedido de sondagem a cada fornecedor e devolvesse o tempo de resposta, o código de estado e o *timestamp* do último sucesso. Esta informação poderia ser exposta numa página de estado no *frontend*, permitindo aos utilizadores compreender se uma falha é devida ao sistema ou a um fornecedor específico, e constituiria um mecanismo de deteção precoce de alterações silenciosas nos contratos das APIs.

7.2.5. Comparação de Cidades

Uma funcionalidade com elevado impacto visual seria a comparação simultânea de duas cidades, apresentando as previsões lado a lado. Esta funcionalidade tem particular relevância no contexto da motivação do projeto comparar as condições em Portugal com as de um país onde residam familiares ou para onde se planeie viajar. A implementação exigiria uma rota dedicada no *frontend* (ex.: `/compare?a=Lisboa&b=Tokyo`) e a execução em paralelo de dois conjuntos de chamadas ao *backend*, uma por cidade. As duas cidades poderiam ser servidas por fornecedores diferentes, já que cada uma seguiria o fluxo normal de seleção de fornecedor.

7.2.6. Análise de Precisão Histórica

O Open-Meteo disponibiliza uma API de dados históricos (`/v1/archive`) com reanálise ERA5 desde 1940. Uma extensão analítica relevante seria a comparação entre as previsões emitidas por cada fornecedor e os dados de reanálise histórica para as mesmas datas e localizações. Esta análise permitiria quantificar a vantagem de precisão dos modelos regionais (IPMA, AEMET, CWA) face ao modelo global Open-Meteo nas respetivas áreas de cobertura, transformando o

sistema de uma plataforma de dados numa plataforma de análise e constituindo uma contribuição académica diferenciadora.

7.2.7. Aplicação Progressiva (*Progressive Web App*)

A transformação do *frontend* numa PWA (*Progressive Web App*) permitiria ao utilizador instalar a aplicação no telemóvel como se fosse uma aplicação nativa, sem necessidade de *app store*. O mecanismo central seria um *service worker* que guardaria em *cache* a última previsão obtida, tornando-a disponível mesmo sem ligação à internet. Dado que a pilha tecnológica já usa *Create React App*, a integração de um *service worker* é nativamente suportada e exigiria alterações mínimas à base de código existente.

8. Referências Bibliográficas

- Adrian, G., Jean, M., Barrell, S., & Riishojgaard, L. P. (2021). WMO Data Policy for the 21st Century. *WMO Bulletin*, 70(2). <https://wmo.int/media/magazine-article/wmo-data-policy-21st-century>
- Agencia Estatal de Meteorología (AEMET). (2024). *AEMET OpenData—API REST*. <https://opendata.aemet.es>
- Bauer, P., Thorpe, A., & Brunet, G. (2015). The Quiet Revolution of Numerical Weather Prediction. *Nature*, 525(7567), 47–55. <https://doi.org/10.1038/nature14956>
- Bengtsson, L., Andrae, U., Aspelien, T., Batrak, Y., Calvo, J., Rooy, W. de, Gleeson, E., Hansen-Sass, B., Homleid, M., Hortal, M., Ivarsson, K.-I., Lenderink, G., Niemelä, S., Nielsen, K. P., Onvlee, J., Rontu, L., Samuelsson, P., Muñoz, D. S., Subias, A., ... Køltzow, M. Ø. (2017). The HARMONIE–AROME Model Configuration in the ALADIN–HIRLAM NWP System. *Monthly Weather Review*, 145(5), 1919–1935. <https://doi.org/10.1175/MWR-D-16-0417.1>
- Bi, K., Xie, L., Zhang, H., Chen, X., Gu, X., & Tian, Q. (2023a). Accurate medium-range global weather forecasting with 3D neural networks. *Nature*, 619, 533–538. <https://doi.org/10.1038/s41586-023-06185-3>
- Bi, K., Xie, L., Zhang, H., Chen, X., Gu, X., & Tian, Q. (2023b). Author Correction: Accurate medium-range global weather forecasting with 3D neural networks. *Nature*, 621(7980), E45–E45. <https://doi.org/10.1038/s41586-023-06545-z>
- Bubnová, R., Hello, G., Bénard, P., & Geleyn, J.-F. (1995). Integration of the Fully Elastic Equations Cast in the Hydrostatic Pressure Terrain-Following Coordinate in the Framework of the ARPEGE/ALADIN NWP System. *Monthly Weather Review*, 123(2), 515–535. [https://doi.org/10.1175/1520-0493\(1995\)123%3C0515:IOTFEE%3E2.0.CO;2](https://doi.org/10.1175/1520-0493(1995)123%3C0515:IOTFEE%3E2.0.CO;2)
- Central Weather Administration (CWA). (2024). *CWA Open Data Platform*. <https://opendata.cwa.gov.tw>

Christie, T. & Encode OSS Ltd. (2024). *HTTPX — A next-generation HTTP client for Python*.

<https://www.python-httpx.org>

Emanuel, K., & Zhang, F. (2016). On the Predictability and Error Sources of Tropical Cyclone

Intensity Forecasts. *Journal of the Atmospheric Sciences*, 73(9), 3739–3747.

<https://doi.org/10.1175/JAS-D-16-0100.1>

Escrich, J. (2023). *Harnessing the Adapter Pattern in Microservice Architectures for Vendor*

Agnosticism. [https://medium.com/@jescrich_57703/harnessing-the-adapter-pattern-](https://medium.com/@jescrich_57703/harnessing-the-adapter-pattern-in-microservice-architectures-for-vendor-agnosticism-debc21d2fe21)

[in-microservice-architectures-for-vendor-agnosticism-debc21d2fe21](https://medium.com/@jescrich_57703/harnessing-the-adapter-pattern-in-microservice-architectures-for-vendor-agnosticism-debc21d2fe21)

European Centre for Medium-Range Weather Forecasts (ECMWF). (2025). *ECMWF's AI*

forecasts become operational. [https://www.ecmwf.int/en/about/media-](https://www.ecmwf.int/en/about/media-centre/news/2025/ecmwfs-ai-forecasts-become-operational)

[centre/news/2025/ecmwfs-ai-forecasts-become-operational](https://www.ecmwf.int/en/about/media-centre/news/2025/ecmwfs-ai-forecasts-become-operational)

Fielding, R. T. (2000). *Architectural Styles and the Design of Network-Based Software*

Architectures [PhD Thesis, University of California, Irvine].

<https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>

Frnda, J., Durica, M., Rozhon, J., Vojtekova, M., Nedoma, J., & Martinek, R. (2022). ECMWF

short-term prediction accuracy improvement by deep learning. *Scientific Reports*,

12(1), 7898. <https://doi.org/10.1038/s41598-022-11936-9>

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable*

Object-Oriented Software. Addison-Wesley.

Giorgi, F. (2019). Thirty Years of Regional Climate Modeling: Where Are We and Where Are We

Going Next? *Journal of Geophysical Research: Atmospheres*, 124(11), 5696–5723.

<https://doi.org/10.1029/2018JD030094>

Git. (s.d.). Obtido 6 de julho de 2026, de <https://git-scm.com/>

Hersbach, H., Bell, B., Berrisford, P., Hirahara, S., Horányi, A., Muñoz-Sabater, J., Nicolas, J.,

Peubey, C., Radu, R., Schepers, D., Simmons, A., Soci, C., Abdalla, S., Abellan, X.,

Balsamo, G., Bechtold, P., Biavati, G., Bidlot, J., Bonavita, M., & Thépaut, J.-N. (2020).

- The ERA5 Global Reanalysis. *Quarterly Journal of the Royal Meteorological Society*, 146(730), 1999–2049. <https://doi.org/10.1002/qj.3803>
- Hsiao, L.-F., Chen, D.-S., Hong, J.-S., Yeh, T.-C., & Fong, C.-T. (2020). Improvement of the Numerical Tropical Cyclone Prediction System at the Central Weather Bureau of Taiwan: TWRF (Typhoon WRF). *Atmosphere*, 11(6), 657.
<https://doi.org/10.3390/atmos11060657>
- Hussain, S., Keung, J., & Khan, A. A. (2017). The Effect of Gang-of-Four Design Patterns Usage on Design Quality Attributes. *Proceedings of the 2017 IEEE International Conference on Software Quality, Reliability and Security (QRS)*, 263–273.
<https://doi.org/10.1109/QRS.2017.37>
- IASA Global. (2023). *Aggregator Pattern*. https://iasa-global.github.io/btabok/aggregator_pattern.html
- Instituto Português do Mar e da Atmosfera (IPMA). (2024a). *IPMA Open Data API*.
<https://api.ipma.pt>
- Instituto Português do Mar e da Atmosfera (IPMA). (2024b). *IPMA Open Data API*.
<https://api.ipma.pt>
- Lam, R., Sanchez-Gonzalez, A., Willson, M., Wirnsberger, P., Fortunato, M., Alet, F., Ravuri, S., Ewalds, T., Eaton-Rosen, Z., Hu, W., Merose, A., Hoyer, S., Holland, G., Vinyals, O., Stott, J., Pritzel, A., Mohamed, S., & Battaglia, P. (2023). Learning skillful medium-range global weather forecasting. *Science*, 382(6677), 1416–1421.
<https://doi.org/10.1126/science.adi2336>
- Mass, C., & Vuckovic, M. (2023). *Implementation of IFS Cycle 48r1*.
<https://confluence.ecmwf.int/display/FUG/Implementation+of+IFS+Cycle+48r1>
- Melgarejo-Hernández, J., García-Tapia-Mateo, P., Morales-García, J., & Mazón, J.-N. (2026). Near-Real-Time Integration of Multi-Source Seismic Data. *Sensors*, 26(2), 451.
<https://doi.org/10.3390/s26020451>

Meta Platforms, Inc. (2024). *React—The library for web and native user interfaces*.

<https://react.dev>

Neovim. (s.d.). Neovim. Obtido 6 de julho de 2026, de <https://neovim.io/>

Prein, A. F., Langhans, W., Fosser, G., Ferrone, A., Ban, N., Goergen, K., Keller, M., Tölle, M.,
Gutjahr, O., Feser, F., Brisson, E., Kollet, S., Schmidli, J., van Lipzig, N. P. M., & Leung, R.

(2015). A Review on Regional Convection-Permitting Climate Modeling:

Demonstrations, Prospects, and Challenges. *Reviews of Geophysics*, 53(2), 323–361.

<https://doi.org/10.1002/2014RG000475>

Rabier, F., Järvinen, H., Klinker, E., Mahfouf, J.-F., & Simmons, A. (2000). The ECMWF

Operational Implementation of Four-Dimensional Variational Assimilation. I:

Experimental Results with Simplified Physics. *Quarterly Journal of the Royal*

Meteorological Society, 126(564), 1143–1170.

<https://doi.org/10.1002/qj.49712656415>

Ramírez, S. (2024). *FastAPI — High Performance, Easy to Learn, Fast to Code, Ready for*

Production. <https://fastapi.tiangolo.com>

Rodríguez, C., Baez, M., Daniel, F., Casati, F., Trabucco, J. C., Canali, L., & Percannella, G. (2016).

REST APIs: A Large-Scale Analysis of Compliance with Principles and Best Practices.

Web Engineering — Proceedings of the 16th International Conference on Web

Engineering (ICWE 2016), Lecture Notes in Computer Science, 9671, 21–39.

https://doi.org/10.1007/978-3-319-38791-8_2

Seity, Y., Brousseau, P., Malardel, S., Hello, G., Bénard, P., Bouttier, F., Lac, C., & Masson, V.

(2011). The AROME-France Convective-Scale Operational Model. *Monthly Weather*

Review, 139(3), 976–991. <https://doi.org/10.1175/2010MWR3425.1>

TatvaSoft. (2023). *Microservices Design Patterns*.

<https://www.tatvasoft.com/outsourcing/2023/11/microservices-design-patterns.html>

venv—Creation of virtual environments. (s.d.). Python Documentation. Obtido 6 de julho de 2026, de <https://docs.python.org/3/library/venv.html>

W3Schools. (2026). *React Hooks*. https://www.w3schools.com/react/react_hooks.asp

Zhou, X., Zhu, Y., Hou, D., Fu, B., Li, W., Guan, H., Sinsky, E., Kolczynski, W., Xue, X., Luo, Y., Peng, J., Yang, B., Tallapragada, V., & Pegen, P. (2022). The Development of the NCEP Global Ensemble Forecast System Version 12. *Weather and Forecasting*, 37(6), 1069–1084. <https://doi.org/10.1175/WAF-D-21-0112.1>

Zippenfenig, Z. (2024). *Open-Meteo—Free Weather API for Non-Commercial Use*. <https://open-meteo.com>

9. Apêndices

Apêndice A – Website Documentação URL

<https://schweizerdiem.github.io/ispgaya-internal-project/>

Apêndice B - Estrutura de ficheiros do *backend*

```
weather-api/  
  
├─ app/  
  
|   ├─ main.py           # App FastAPI, CORS, registo de routers  
  
|   ├─ config.py         # Chaves de API carregadas de .env  
  
|   └─ geocoding.py      # Resolução de nome de cidade → coordenadas  
  
├─ models/  
  
|   ├─ open_meteo/schemas.py  
  
|   ├─ aemet/schemas.py  
  
|   ├─ ipma/schemas.py  
  
|   └─ cwa/schemas.py  
  
├─ providers/  
  
|   ├─ open_meteo/adapter.py  
  
|   ├─ aemet/adapter.py  
  
|   ├─ aemet/municipios.py  
  
|   ├─ ipma/adapter.py  
  
|   ├─ ipma/cities.py  
  
|   ├─ cwa/adapter.py  
  
|   └─ cwa/locations.py  
  
├─ routers/  
  
|   ├─ open_meteo/city.py  
  
|   └─ aemet/forecast.py
```

```

|   └─ ipma/forecast.py
|   └─ cwa/forecast.py
└─ geo.py

```

Apêndice C - Estrutura de ficheiros do *frontend*

```
weather-frontend/src/
```

```

└─ App.jsx                # Componente raiz, estado de vista
└─ components/
|   └─ LocationPicker.jsx  # Barra de pesquisa com dropdown
|   └─ CurrentCard.jsx     # Condições atuais
|   └─ HourlyForecast.jsx  # Previsão horária
|   └─ DailyForecast.jsx   # Previsão diária
|   └─ TemperatureChart.jsx # Gráficos MUI X Charts
|   └─ ProviderNote.jsx    # Nota informativa do fornecedor
|   └─ RecentSearches.jsx  # Histórico de pesquisas
|   └─ UnitsToggle.jsx     # Toggles °C/°F, km/h/mph e idioma
|   └─ AboutPage.jsx       # Página About
└─ hooks/
|   └─ useWeather.js       # Lógica de fetch e seleção de provider
|   └─ useGeolocation.js   # Geolocalização GPS
└─ context/
|   └─ UnitsContext.jsx    # Unidades de medida globais
|   └─ LanguageContext.jsx # Idioma global (EN/PT)
└─ utils/
|   └─ api.js              # Todas as chamadas ao backend
|   └─ useLocations.js     # Pesquisa unificada multi-provider
|   └─ locations.js        # Lista estática Open-Meteo
|   └─ constants.js        # Flags e metadados de provider

```

| └─ recents.js

Gestão do histórico (localStorage)

|─ i18n/

└─ translations.js

Strings EN/PT